

A structured quasi-Newton augmented Lagrangian algorithm for nonlinear least-squares with linear and nonlinear constraints

Pierre Borie ^{*} Fabian Bastin ^{*}
`pierre.borie@umontreal.ca` `bastin@iro.umontreal.ca`

Stéphane Dellacherie ^{†‡}
`dellacherie.stephane@hydroquebec.com`

September 15, 2026

Abstract

We present an algorithm for solving nonlinear least-squares problems subject to a mix of nonlinear constraints, linear equalities, and bounds. The nonlinear constraints are handled through an augmented Lagrangian reformulation of the objective, while the linear constraints are kept explicit. Each outer iteration therefore requires the approximate minimization of the augmented Lagrangian over a polyhedron, which we carry out with a gradient projection technique combined with a trust-region step computation. To capture the second-order information specific to the least-squares structure, we approximate the Hessian of the augmented Lagrangian by a Gauss–Newton term corrected by a structured quasi-Newton update, activated by a hybrid switching rule. The correction is built from a secant equation that we derive for the augmented Lagrangian itself. As a by-product of the analysis, we isolate the properties of the projected gradient path over a general polyhedron that make Cauchy-point-based trust-region methods applicable there, a result independent of the least-squares setting. We establish global convergence of both the inner minimization and the overall method to a first-order critical point, and report numerical experiments. Our method outperforms an augmented Lagrangian solver built on the same outer iteration but without least-squares structure, and is competitive in computation time with a state-of-the-art solver for nonlinear programming.

Keywords: constrained nonlinear least-squares, augmented Lagrangian method, structured quasi-Newton update.

^{*}Université de Montréal, Department of Computer Science and Operations Research, Montreal, QC, Canada

[†]Hydro-Québec, Montreal, QC, Canada

[‡]Université du Québec à Montréal, Montreal, QC, Canada

1 Introduction

We consider the constrained nonlinear least-squares (NLS) problem

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & \frac{1}{2} \|r(x)\|^2 \\ \text{s.t.} \quad & c(x) = 0 \\ & x \in \Omega, \end{aligned} \tag{1.1}$$

where $\|\cdot\|$ denotes the ℓ_2 norm. The residual and constraint functions $r: \mathbb{R}^n \rightarrow \mathbb{R}^{n_r}$ and $c: \mathbb{R}^n \rightarrow \mathbb{R}^{n_c}$ are assumed to be twice continuously differentiable with respective Jacobians J and C . We may also write the least-squares objective as a general function f with gradient

$$\nabla f(x) = J(x)^\top r(x).$$

The set Ω consists of linear constraints and is defined as

$$\Omega = \{x \in \mathbb{R}^n \mid Ax = b, l \leq x \leq u\}, \tag{1.2}$$

with $A \in \mathbb{R}^{m \times n}$, $m < n$, $b \in \mathbb{R}^m$, and $l, u \in \mathbb{R}^n$. Without loss of generality, some components of the latter two vectors can be set to $\pm\infty$ for unbounded parameters. For convenience, we also assume that the linear equalities do not involve single-variable constraints. Formulation (1.1) is also suitable for problems with nonlinear inequality constraints of the form $g(x) \geq 0$. The latter are transformed into equality constraints by adding non-negative slack variables, which gives the new constraints

$$g(x) - \nu = 0, \quad \nu \geq 0.$$

The lower and upper bounds associated with these slack variables are thus 0 and ∞ respectively. Linear inequality constraints can be converted into equalities similarly.

The Lagrangian for problem (1.1), with respect to the nonlinear constraints, is defined by

$$\mathcal{L}(x, \lambda) = f(x) + \langle \lambda, c(x) \rangle,$$

where $\lambda \in \mathbb{R}^{n_c}$ is the vector of Lagrange multipliers and $\langle \cdot, \cdot \rangle$ is the standard inner product.

An algorithm for solving problem (1.1) aims to find a first-order critical point (x_*, λ_*) that satisfies the necessary condition

$$\begin{aligned} x_* &\in \Omega \text{ and } c(x_*) = 0 \\ P_\Omega [x_* - \nabla_x \mathcal{L}(x_*, \lambda_*)] &= x_*, \end{aligned}$$

where $P_\Omega [\cdot]$ is the projection operator onto Ω , defined, for any vector x , as

$$P_\Omega [x] = \operatorname{argmin}_{v \in \Omega} \|v - x\|^2.$$

The NLS structure arises in data fitting, parameter estimation, and scientific computing, and its efficient resolution has attracted a sustained research effort [13, 25, 48]. A distinctive feature of NLS problems is that the Hessian of the objective decomposes into a first-order part $J(x)^\top J(x)$, where $J(x)$ is the Jacobian of r , and a second-order part involving the individual Hessians of the residuals weighted by their values. Exploiting this decomposition is the central challenge in designing efficient algorithms. The Gauss–Newton (GN) method retains only the

first-order term, which is effective when residuals are small at the solution but can lead to poor convergence otherwise. The Levenberg–Marquardt (LM) method, initiated by Levenberg [38] and Marquardt [44] and further studied, for instance, in [7, 9], regularizes the GN matrix by a scalar multiple of the identity, providing robustness at the cost of potentially slower convergence on large-residual problems. A richer class of approaches, known as structured quasi-Newton (SQN) methods, approximates the second-order part through secant equations tailored to the least-squares structure. The NL2SOL package [26, 37] is a popular implementation of this idea, complementing the GN term with a structured quasi-Newton correction updated via a dedicated secant equation. This algorithm also switches between the GN model and the full Hessian depending on the quality of the step. Other hybrid strategies with a switching criterion based on the residual magnitude have also been explored in [1, 28, 65]. Further SQN formulations, including factorized updates and sizing strategies, have been proposed in [4, 10, 36, 63, 64]. A comprehensive numerical comparison of these techniques can be found in [42].

Extending structured NLS methods to the constrained setting has received comparatively less attention, though it remains an active area of research. Such formulations arise, for example, in data-fitting problems where the constraints enforce the physical consistency of an underlying model [24, 33, 53]. The convex case, especially the bound-constrained case, is studied in [5, 6, 31, 51].

For more general nonlinear constraints, most methods operate within a sequential quadratic programming (SQP) framework. In that paradigm, each iteration solves a quadratic subproblem obtained by linearizing the constraints and approximating the Hessian of the Lagrangian. An SQP method whose Hessian approximation is based on a secant equation derived from that of Biggs [4] is proposed by Li et al. [39]. A BFGS update of the projected Hessian of a merit function is used in [60], while a structured approximation of the projected Hessian is updated within the null space of the active constraints in [43]. For problems with equality constraints only, more recent work includes regularized approaches. A Levenberg–Marquardt approximation of the Lagrangian Hessian is combined with a trust-region method in [9], and a regularized KKT system is derived in [50] by adding primal and dual regularization terms. Structured approximations of the Hessian within general constrained formulations have also been studied from a theoretical perspective in [59]. When the constraints can be split into two categories, as in formulation (1.1), an alternative to SQP methods is the augmented Lagrangian (AL) framework, also known as the method of multipliers, independently introduced by Hestenes [34] and Powell [52], with further theoretical foundations provided by Rockafellar [54]. Rather than solving a sequence of quadratic subproblems, AL methods incorporate the nonlinear constraints into the objective as a penalty term and solve a sequence of subproblems in which only the easier constraints remain. For problem (1.1), this amounts to approximately minimizing, at each outer iteration, the AL function over the polyhedral set Ω , thereby reducing the problem to a linearly constrained one. This strategy is particularly attractive as the linear part can be handled directly and efficiently by gradient projection techniques, without sacrificing the ability to enforce nonlinear constraints. The theoretical underpinnings of this approach, including global convergence results, are developed in [15, 17, 20, 21] and summarized in [22, Chapter 14]. A major implementation of this philosophy is the LANCELOT solver [19], which handles bound-constrained subproblems using a gradient projection technique [16]. More recent AL implementations have broadened this framework in several directions. Convergence is established under weaker constraint qualifications in [2]. A primal-dual AL method that behaves like a stabilized SQP method in the local regime is proposed in [29], a matrix-free AL approach is implemented in [3], and an adaptive AL algorithm with inexact subproblem solves

is developed in [23]. For the inner minimization that arises at each outer AL iteration, gradient projection methods [40, 46] are particularly effective for bound-constrained problems as they allow for rapid changes in the active set throughout the iterations. The linear equality case can be handled efficiently, as the projections can be computed by solving a system of normal equations [32]. When both bounds and linear equalities are present, as in our setting, direct projection onto Ω is not available in closed form. Nevertheless, we will see in the next section that the norm of the reduced gradient provides a practical criticality measure, as used in solvers such as MINOS [47] and LSNNO [61].

It is worth pointing out what this combination leaves uncovered in the existing literature. Structured quasi-Newton methods for least-squares are, with the exceptions discussed above, developed for the unconstrained case. Augmented Lagrangian solvers such as LANCELOT [19] and Percival [3] handle bound-constrained subproblems but treat the objective as a generic nonlinear function. The structured methods available for constrained least-squares operate within an SQP framework [39, 43, 60] and do not keep bounds and linear equalities explicit in their subproblems. Among the methods surveyed above, we are not aware of one combining the three features at once, namely an approximation exploiting the least-squares structure, a feasible set mixing nonlinear equalities with a general polyhedron whose linear constraints are never penalized nor linearized, and a supporting global convergence analysis. It is this combination that our algorithm addresses.

The present work brings the structured quasi-Newton philosophy of nonlinear least-squares into the augmented Lagrangian framework. Our contributions are the following.

- (i) We derive a structured secant equation for the Hessian of the AL, given in (2.26), which simultaneously captures the curvature carried by the residuals and that carried by the nonlinear constraints, the latter being weighted by the shifted multipliers $\bar{\lambda}$ produced by the outer iteration. We show that correcting the Gauss–Newton term of the AL by a structured update satisfying this equation yields an approximation that also satisfies the standard secant equation, so that the structure is exploited without sacrificing the usual quasi-Newton properties. To the best of our knowledge, this is the first structured quasi-Newton approximation designed for the AL of a least-squares problem. We complement it with a hybrid switching rule driven by the relative decrease of the AL itself, rather than of the objective, so that the decision to activate the second-order correction reflects the penalized function that is actually minimized.
- (ii) We extend the gradient projection inner minimization of [16], formulated for bound constraints and underlying the LANCELOT solver, to the general polyhedron (1.2) in which bounds and linear equalities coexist. The projection onto Ω is then no longer available in closed form, and the piecewise structure of the projected gradient path, on which both the Cauchy point computation and the convergence argument rest, has to be re-established. We show that the path remains piecewise linear with nested active sets, and we derive an explicit procedure computing its first local minimizer in that setting (Section 2.2 and Appendix A).
- (iii) We establish global convergence of the resulting method to a first-order critical point of problem (1.1). The analysis covers a feasible set combining nonlinear equalities with a general polyhedron, for which the constraint qualification has to be restated, and it only requires mild growth conditions on the Hessian model, so that it applies to every variant we implement.

- (iv) We provide TRAULLS, an open-source Julia [11] implementation available at <https://github.com/UncertainLab/Traulls.jl>. All the experiments of Section 4 can be reproduced from the `benchmark` folder of that repository.

Together, these ingredients yield a solver specifically designed for NLS problems while handling constraints of general form, including nonlinear equalities and inequalities alongside linear equalities and bounds.

We end this section by introducing some notation. Components of a vector $x \in \mathbb{R}^n$ are denoted x_i for $i = 1, \dots, n$. Vectors e_i denote the columns of the $n \times n$ identity matrix written I_n – or simply I when the dimension is clear from the context. When describing iterative processes to solve problem (1.1), we index vectors and matrices by the iteration number. Our method has a nested structure (Section 2): we use an uppercase index K for its outer iterations and a lowercase index k for the inner iterations. For instance, at outer iteration K , (x_K, λ_K) is the current primal-dual iterate, while for functions evaluated at an iterate x_k we use the shorthand notation $r_k = r(x_k)$, $c_k = c(x_k)$, $J_k = J(x_k)$, etc., and similarly at outer iterates. To avoid interfering with the previous notation for components, we add parentheses to make the distinction clear. For instance, $(x_k)_i$ denotes the i -th component of vector x_k . For any sequence $(u_k)_k$, the limit of a converging subsequence indexed by $\mathcal{K} \subset \mathbb{N}$ will be written $\lim_{k \in \mathcal{K}} u_k$. The orthogonal complement of a subspace $V \subset \mathbb{R}^n$, i.e., the set $\{w \mid \langle w, v \rangle = 0 \text{ for all } v \in V\}$, will be written $V^\perp$.

The rest of the paper is organized as follows. In Section 2, we present our algorithmic framework and discuss implementation details. We study global convergence in Section 3. Numerical experiments are shown in Section 4 and we conclude with perspectives.

2 The algorithm

2.1 Augmented Lagrangian framework

We formally state the first assumptions about problem (1.1).

Assumption 1. *Residuals r and constraints c are twice continuously differentiable on $\mathbb{R}^n$.*

Assumption 2. *The equality constraint matrix A is full row rank.*

Assumption 3. *The feasible region for the linear constraints Ω is non-empty.*

Our method is based on the augmented Lagrangian (AL) function defined as

$$\Phi(x, \lambda, \mu) := \frac{1}{2} \|r(x)\|^2 + \langle \lambda, c(x) \rangle + \frac{\mu}{2} \|c(x)\|^2, \quad (2.1)$$

where $\mu > 0$ is a penalty parameter.

We maintain the linear constraints in the formulation of the problem and only penalize the violation of the nonlinear constraints. The gradient is given by

$$\nabla_x \Phi(x, \lambda, \mu) = J(x)^\top r(x) + C(x)^\top \bar{\lambda}(x, \lambda, \mu),$$

where

$$\bar{\lambda}(x, \lambda, \mu) := \lambda + \mu c(x), \quad (2.2)$$

are the first-order estimates of the Lagrange multipliers. The AL and Lagrangian gradients are related by the identity

$$\nabla_x \Phi(x, \lambda, \mu) = \nabla_x \mathcal{L}(x, \bar{\lambda}(x, \lambda, \mu)). \quad (2.3)$$

The Hessian is given by

$$\nabla_{xx}^2 \Phi(x, \lambda, \mu) = J(x)^\top J(x) + \mu C(x)^\top C(x) + S(x, \lambda, \mu)$$

with the second-order terms explicitly given by

$$S(x, \lambda, \mu) = \sum_{i=1}^{n_r} r_i(x) \nabla^2 r_i(x) + \sum_{i=1}^{n_c} \nabla^2 c_i(x) \bar{\lambda}_i(x, \lambda, \mu). \quad (2.4)$$

For fixed λ and μ , reformulating problem (1.1) with function (2.1) gives the linearly constrained problem

$$\begin{aligned} \min_x \quad & \Phi(x, \lambda, \mu) \\ \text{s.t.} \quad & x \in \Omega. \end{aligned} \quad (2.5)$$

Moving the nonlinear constraints into the objective simplifies the problem since only linear constraints are left, which enables one to use iterative methods for linearly constrained optimization [16, 32] while still improving feasibility of the nonlinear constraints. Since Ω is a convex set, a first-order critical point of (2.5) can also be characterized by the condition

$$P_\Omega [x_* - \nabla_x \Phi(x_*, \lambda, \mu)] = x_*.$$

In our methods, and as in standard AL methods [17, 22], each iterate x_K is an approximate solution of a subproblem of the form (2.5) satisfying

$$\|P_\Omega [x_K - \nabla_x \Phi(x_K, \lambda_K, \mu_K)] - x_K\| \leq \omega_K, \quad (2.6)$$

for a fixed vector of multipliers λ_K , penalty parameter μ_K and tolerance ω_K . If the iterate x_K improves the feasibility violation of the nonlinear constraints, a dual ascent step is taken on the Lagrange multipliers by setting

$$\lambda_{K+1} = \lambda_K + \mu_K c_K = \bar{\lambda}(x_K, \lambda_K, \mu_K).$$

If, on the contrary, feasibility has not been improved, the minimization (2.5) is restarted using an increased penalty parameter $\mu_{K+1} > \mu_K$ to enforce feasibility. The method is outlined in Algorithm 1. Following our notation for iteration-dependent quantities, we respectively write $\nabla_x \Phi(x_K, \lambda_K, \mu_K)$ and $\bar{\lambda}(x_K, \lambda_K, \mu_K)$ in the compact form $\nabla_x \Phi_K$ and $\bar{\lambda}_K$. One first notices that Algorithm 1 has an outer-inner iteration structure, in the sense that each iteration requires approximately solving an optimization problem (line 3), which also involves an iterative process. The techniques used for the latter are detailed in the following sections.

Tolerances in Algorithm 1 are updated in such a way that both sequences ω_K and η_K tend to 0 when $K \rightarrow \infty$. The update scheme that we outline follows the rules described in [22, Chapter 14]. The parameter values used in our implementation are given in Section 2.5.

We describe the convergence in terms of the norm of the projected gradient, but this quantity is not directly computable in practice, as there is no closed-form formula for the projection on a polyhedral set of the form Ω . For our implementation, we will use the norm of the reduced gradient, which we define now.

Algorithm 1 Augmented Lagrangian algorithm

Require: Initial point $x_0^s \in \Omega$, multiplier estimate λ_0 , penalty parameter $\mu_0 > 0$, increase factor $\tau > 1$, tolerance update constants $\omega, \eta, \kappa_\omega, \kappa_\eta, \beta_\omega, \beta_\eta > 0$, stopping tolerances $\omega_*, \eta_* > 0$

- 1: Set initial tolerances $\omega_0 \leftarrow \omega \mu_0^{-\kappa_\omega}$ and $\eta_0 \leftarrow \eta \mu_0^{-\kappa_\eta}$
- 2: **for** $K=0,1,2,\dots$ **do**
- 3: Starting from x_K^s , approximately solve $\min_{x \in \Omega} \Phi(x, \lambda_K, \mu_K)$ to find $x_K \in \Omega$ satisfying (2.6).
- 4: **if** $\|c(x_K)\| \leq \eta_K$ **then**
- 5: **if** $\|P_\Omega[x_K - \nabla_x \Phi_K] - x_K\| \leq \omega_*$ and $\|c(x_K)\| \leq \eta_*$ **then**
- 6: **stop** and **return** approximate solution (x_K, λ_K)
- 7: **end if**
- 8: Update the Lagrange multipliers $\lambda_{K+1} \leftarrow \bar{\lambda}_K$
- 9: Set the next starting point $x_{K+1}^s \leftarrow x_K$
- 10: Leave the penalty parameter unchanged $\mu_{K+1} \leftarrow \mu_K$
- 11: Decrease the tolerances $\omega_{K+1} \leftarrow \omega_K \mu_{K+1}^{-\beta_\omega}$ and $\eta_{K+1} \leftarrow \eta_K \mu_{K+1}^{-\beta_\eta}$
- 12: **else**
- 13: Leave the iterate unchanged $(x_{K+1}^s, \lambda_{K+1}) \leftarrow (x_K^s, \lambda_K)$
- 14: Increase the penalty parameter $\mu_{K+1} \leftarrow \tau \mu_K$
- 15: Update tolerances $\omega_{K+1} \leftarrow \omega \mu_{K+1}^{-\kappa_\omega}$ and $\eta_{K+1} \leftarrow \eta \mu_{K+1}^{-\kappa_\eta}$
- 16: **end if**
- 17: **end for**

For $x \in \Omega$, $\mathcal{A}^+(x)$ and $\mathcal{A}^-(x)$ respectively denote the indices of upper and lower bounds satisfied as equalities (active) at x , and we set $\mathcal{A}(x) := \mathcal{A}^+(x) \cup \mathcal{A}^-(x)$, so that

$$i \in \mathcal{A}(x) \iff x_i \in \{l_i, u_i\}.$$

We also introduce the notion of tangent space at a point x , written $T(x)$ and defined as the set of directions d such that the same constraints are satisfied with equality at both x and $x+d$. In our case, this corresponds to the linear equality constraints and the active bounds. Formally,

$$T(x) := \{d \in \mathbb{R}^n \mid Ad = 0, d_i = 0 \text{ for all } i \in \mathcal{A}(x)\}. \quad (2.7)$$

At a given point (x, λ) and parameter μ , the reduced gradient is defined as

$$P_{T(x)} [\nabla_x \Phi(x, \lambda, \mu)],$$

where $P_{T(x)}[\cdot]$ denotes the orthogonal projection operator onto the tangent space $T(x)$. Provided that the multipliers associated with the active bounds are of appropriate sign, the quantity $\|P_{T(x)} [\nabla_x \Phi(x, \lambda, \mu)]\|$ is an appropriate criticality measure used in optimization algorithms for linearly constrained optimization such as MINOS [47] and LSNNO [61]. Projections on the tangent space also have the advantage of being available in closed form. The subspace $T(x)$ is the null space of the matrix

$$\tilde{A} = \begin{pmatrix} A \\ E_{\mathcal{A}}^\top \end{pmatrix}, \quad (2.8)$$

where $E_{\mathcal{A}}$ is formed by the columns $(e_i)_{i \in \mathcal{A}(x)}$ of the identity. We assume throughout that these active constraints remain non-degenerate.

Assumption 4. *For every $x \in \Omega$, the matrix (2.8) formed from the bounds active at x has full row rank.*

Assumption 4 strengthens Assumption 2: it is a linear independence (non-degeneracy) condition on the active linear constraints, which holds, in particular, when no active bound is linearly dependent on the rows of A . Under this assumption, projecting on $T(x)$ is equivalent to applying the orthogonal projection operator

$$\tilde{P} = I - \tilde{A}^\top \left(\tilde{A} \tilde{A}^\top \right)^{-1} \tilde{A}. \quad (2.9)$$

We will provide more details on how the projections are calculated in practice in Section 2.5, but this offers an initial justification for using

$$\|P_{T_K} [\nabla_x \Phi_K]\| \leq \omega_K \quad (2.10)$$

as a stopping condition in Algorithm 1 instead of (2.6).

2.2 Inner minimization process

In this section, we describe the inner minimization process used to produce the outer iterates of Algorithm 1. Since the current multiplier estimates λ and the penalty parameter μ are fixed, the objective function for the inner minimization only depends on x so we use the shorthand notation $\varphi : x \mapsto \Phi(x, \lambda, \mu)$. Given a point $x_0 \in \Omega$, multiplier estimates λ and a penalty parameter μ , we aim to find an approximate solution of the problem

$$\begin{aligned} \min_x \quad & \varphi(x) \\ \text{s.t.} \quad & x \in \Omega, \end{aligned} \quad (2.11)$$

and such that

$$\|P_{T(x)} [\nabla \varphi(x)]\| \leq \omega,$$

for a tolerance $\omega > 0$. This task also involves an iterative process, for which the iterates will be indexed with subscript k but are distinct from the outer iterates of Algorithm 1. At each iteration k , we compute a step s_k and recursively update the iterate by $x_{k+1} = x_k + s_k$. The step is obtained after minimizing a model of the objective function φ around x_k . We consider the quadratic model

$$m_k(x) = \varphi_k + \langle g_k, x - x_k \rangle + \frac{1}{2} \langle x - x_k, H_k(x - x_k) \rangle, \quad (2.12)$$

where $\varphi_k := \varphi(x_k)$, $g_k := \nabla \varphi_k$ and H_k is a symmetric approximation of the true Hessian $\nabla_{xx}^2 \varphi_k$. Using the latter would negatively impact the performance of the algorithm since computing the second-order terms (2.4) requires too much time and storage in practice. We discuss the aspects of the methods relative to the choice of approximation in Section 2.4. Model (2.12) is appropriate to describe the algorithm in terms of the iterate. When discussing subproblems, we prefer to emphasize the step and would then use the following model of the primal reduction $\varphi(x_k + s) - \varphi_k$, given by

$$q_k(s) = \frac{1}{2} \langle s, H_k s \rangle + \langle g_k, s \rangle.$$

The two formulations are related by $q_k(x - x_k) = m_k(x) - m_k(x_k)$.

We seek to compute the step s_k as an approximate solution of the program

$$\begin{aligned} \min_s \quad & q_k(s) \\ \text{s.t.} \quad & x_k + s \in \Omega \\ & \|s\|_\infty \leq \Delta_k. \end{aligned}$$

Here, s denotes the unknown of the subproblem, whose solution s_k is the step used to compute the new iterate $x_{k+1} = x_k + s_k$.

Since x_0 is feasible, the constraints $x_k + s \in \Omega$ are satisfied at every iteration as long as the steps satisfy

$$As = 0, \quad l - x_k \leq s \leq u - x_k,$$

for all k .

In our method, we also incorporate a trust-region strategy to control and assess the quality of a step. Therefore, we add to each subproblem the constraint $\|s\|_\infty \leq \Delta_k$ with a radius $\Delta_k > 0$, which reflects the domain on which we believe the model approximates the true function well. We use the ℓ_∞ norm because the intersection of the resulting trust region with Ω is again a polyhedron of the form (1.2): the radius constraint only tightens the bounds and leaves the linear equalities untouched, so the subproblem retains the structure of a problem with bounds and linear equalities that our projection machinery handles directly. Since $\|s\|_\infty \leq \Delta_k$ amounts to imposing $s_i \in [-\Delta_k, \Delta_k]$ for all i , we can rewrite this subproblem

$$\min_s \quad q_k(s) \tag{2.13a}$$

$$\text{s.t.} \quad As = 0 \tag{2.13b}$$

$$l_k \leq s \leq u_k, \tag{2.13c}$$

where l_k and u_k are iteration-dependent bounds having components $(l_k)_i = \max(-\Delta_k, l_i - (x_k)_i)$ and $(u_k)_i = \min(\Delta_k, u_i - (x_k)_i)$ for all $i = 1, \dots, n$.

The success of an iteration and the update of the trust region are evaluated by the ratio

$$\rho_k = \frac{\varphi(x_k + s_k) - \varphi_k}{q_k(s_k)}. \tag{2.14}$$

We follow the standard step acceptance criteria [22], which require constants $\eta_1, \eta_2, \gamma_1, \gamma_2$ such that

$$0 < \eta_1 \leq \eta_2 < 1 \text{ and } 0 < \gamma_1 \leq \gamma_2 < 1. \tag{2.15}$$

If $\rho_k > \eta_1$, the step is accepted and the trust region is expanded. Otherwise, the step is rejected and the region reduced. A typical scheme to update the radius Δ_k would be to set

$$\Delta_{k+1} \in \begin{cases} [\Delta_k, \infty) & \text{if } \rho_k \geq \eta_2 \\ [\gamma_2 \Delta_k, \Delta_k] & \text{if } \rho_k \in [\eta_1, \eta_2) \\ [\gamma_1 \Delta_k, \gamma_2 \Delta_k] & \text{if } \rho_k < \eta_1 \end{cases} \tag{2.16}$$

The procedure employed to solve subproblem (2.13) is described in Algorithm 2 and the latter is analyzed in the rest of this section.

Algorithm 2 Trust region inner iteration algorithm

Require: Initial point $x_0 \in \Omega$, radius $\Delta_0 > 0$, constants $\eta_1, \eta_2, \gamma_1, \gamma_2$ satisfying conditions (2.15).

- 1: Set $k \leftarrow 0$
 - 2: **repeat**
 - 3: Compute the model m_k
 - 4: Compute a step s_k that sufficiently reduces the model
 - 5: Compute the ratio ρ_k (2.14)
 - 6: **if** $\rho_k > \eta_1$ **then** set $x_{k+1} \leftarrow x_k + s_k$
 - 7: **else** set $x_{k+1} \leftarrow x_k$
 - 8: **end if**
 - 9: Set Δ_{k+1} according to (2.16)
 - 10: Increment $k \leftarrow k + 1$
 - 11: **until** $\|P_{T_k}[g_k]\| \leq \omega$
-

2.3 Step computation

We consider the current feasible iterate x_k and its associated approximate quadratic model m_k (2.12). The step is computed in two phases.

Cauchy step We first compute a Cauchy step s_k^C that ensures a decrease of the objective function sufficient to establish global convergence of the inner minimization algorithm. Next, we further minimize the objective function by exploring the subspace defined by the constraints active at the Cauchy point $x_k^C := x_k + s_k^C$. This approach is commonly used in gradient projection techniques [48, Chapter 16] and there are different strategies to compute a Cauchy point [16, 40, 46]. Because of the structure of the linear constraints, we cannot directly project on the whole set Ω in practice, but we can exploit prior knowledge of the active bounds to compute the projection iteratively. That is the reason why we compute our Cauchy step by finding the first local minimizer of the model along the projected gradient path

$$s_k(t) = P_\Omega[x_k - tg_k] - x_k \text{ for } t \geq 0. \quad (2.17)$$

Our procedure – described in Appendix A – is an adaptation to the polyhedral case of algorithm SBMIN [16] used for the inner minimization phase of the LANCELOT solver [19].

Subspace minimization In order to improve the rate of convergence of the algorithm, we want the total step to achieve a better reduction than the Cauchy step. To do so, we build the next iterate x_{k+1} after a finite sequence of M minor iterates $x_{k,1}, \dots, x_{k,M+1}$. The sequence starts at the Cauchy point and ends at the next iterate, i.e., $x_k + s_k^C = x_{k,1}$ and $x_{k+1} = x_{k,M+1}$. This type of approach has been shown to be effective for general optimization with bound constraints [40] and has also been incorporated into other AL algorithms for the inner loop minimization [3, 19].

Each minor iterate is defined after the previous one and is decomposed into

$$x_{k,j+1} = x_{k,j} + w_{k,j},$$

where $w_{k,j}$ is a descent direction for the quadratic model q_k . For each minor iterate, we require

$$x_{k,j} \in \Omega, \quad \|x_{k,j} - x_k\|_\infty \leq \Delta_k, \quad \mathcal{A}(x_k^C) \subseteq \mathcal{A}(x_{k,j}). \quad (2.18)$$

The first two conditions are merely that each minor iterate is feasible and the associated step lies within the trust region, while the last condition means that we can only add active bounds during this process. Note that in this context, a bound can become active with respect to the trust region and not only the original bounds on the variables. We also require the sufficient decrease between two successive minor iterates

$$m_k(x_{k,j+1}) \leq m_k(x_{k,j}), \quad j = 1, \dots, M. \quad (2.19)$$

After each minor iteration, the corresponding step is $s_{k,j} := x_{k,j} - x_k$.

The search direction $w_{k,j}$ is an approximate minimizer of the subproblem

$$\min_w m_k(x_{k,j} + w) \quad (2.20a)$$

$$\text{s.t.} \quad Aw = 0 \quad (2.20b)$$

$$w_i = 0, \quad i \in \mathcal{A}(x_{k,j}). \quad (2.20c)$$

At a minor iteration, the free variables, indexed by $\mathcal{F}(x_{k,j})$, are implicitly subject to the bounds

$$\left(l^{(k,j)}\right)_i \leq w_i \leq \left(u^{(k,j)}\right)_i, \quad i \in \mathcal{F}(x_{k,j}) \quad (2.21)$$

where $l^{(k,j)} := l_k - s_{k,j}$ and $u^{(k,j)} := u_k - s_{k,j}$.

The minor subproblem (2.20) is solved by applying the projected conjugate gradient [32] method with the previous minor iterate $x_{k,j}$ as a starting point. Three termination cases can occur. First, a direction can be generated such that one of the bounds (2.21) is violated for one of the free variables. When this happens, the search direction is scaled so that the associated component lies at the bound and the CG iterations stop. The second case occurs when we generate a direction of negative curvature and is handled similarly to the first one, i.e., we modify the direction so that all the variables remain within their bounds. The third case is the normal termination one and occurs when a local minimizer – with respect to a given tolerance – has been found. In all cases, we return the obtained search direction $w_{k,j}$ and perform the projected line search to compute the point $x_{k,j+1}$ such that eqs. (2.18) and (2.19) are verified. The handling of the termination cases differs in the continuation of the procedure. The last two cases (negative curvature and local optimality) terminate the minor iteration loop. On the contrary, if the CG iterations stopped because a bound was hit, we go back to solving (2.20) using the projected conjugate gradient method with $x_{k,j+1}$ as a starting point and $\mathcal{A}(x_{k,j+1}) \supset \mathcal{A}(x_{k,j})$ as a new set of fixed components. The specification of the projected conjugate gradient method applied to solving (2.20) is outlined in Algorithm 3. The operations occurring at lines 1 and 16 are projections onto the null space defined by the constraints (2.20b)–(2.20c).

An interesting feature would be to complement each search direction by a steplength computed by a projected search [40, 46]. This allows one to add more than one constraint at each minor iteration to the active set. The downside is that it would also require computing the projection of the gradient direction (2.17) on the feasible set Ω at every new trial value of the steplength. When Ω only contains bound constraints, the projection is trivial and cheap to compute. The linear equality case is more costly but the projections can also be computed directly. When Ω is of the form (1.2), the projection is less trivial, as it amounts to solving a minimum-distance quadratic program over Ω at every trial steplength. Performing such a projection repeatedly would require a dedicated quadratic programming solver – an external

Algorithm 3 The projected conjugate gradient method applied to (2.20)

```

1: Set  $w \leftarrow 0$ ,  $r \leftarrow H_k(x_{k,j} - x_k) + g_k$ ,  $v \leftarrow \tilde{P}r$ ,  $p \leftarrow -v$ 
2: Set tolerance  $\varepsilon_{\text{cg}} \leftarrow \kappa_{\text{cg}} \|v\|$ 
3: repeat
4:   if  $\langle p, H_k p \rangle \leq 0$  then
5:     Set  $w^+ \leftarrow w + \gamma p$  where  $\gamma$  is the smallest factor such that  $w_i^+ \in \{l_i^{(k,j)}, u_i^{(k,j)}\}$  for
      $i \in \mathcal{F}(x_{k,j})$ 
6:     STOP
7:   end if
8:   Set  $\alpha \leftarrow \langle r, v \rangle / \langle p, H_k p \rangle$ 
9:   Set  $w^+ \leftarrow w + \alpha p$ 
10:  if  $w^+$  violates a bound then
11:     $w^+ \leftarrow w + \gamma p$  where  $\gamma$  is the smallest factor such that  $w_i^+ \in \{l_i^{(k,j)}, u_i^{(k,j)}\}$ 
12:    for  $i \in \mathcal{F}(x_{k,j})$ 
13:    STOP
14:  end if
15:  Set  $r^+ \leftarrow r + \alpha H_k p$ 
16:  Set  $v^+ \leftarrow \tilde{P}r^+$ 
17:  if  $\sqrt{\langle r^+, v^+ \rangle} < \varepsilon_{\text{cg}}$  then STOP
18:  end if
19:  Set  $\beta \leftarrow \langle r^+, v^+ \rangle / \langle r, v \rangle$ 
20:  Set  $p \leftarrow -v^+ + \beta p$ 
21:  Set  $w \leftarrow w^+$ ,  $r \leftarrow r^+$ ,  $v \leftarrow v^+$ 
22: until  $2(n - m - |\mathcal{A}(x_{k,j})|)$  iterations have been done
23: return  $w^+$ 

```

dependency whose per-trial cost would outweigh the benefit of enlarging the active set more aggressively – which we prefer to avoid.

We stop the minor-iterations procedure when the reduced gradient associated with the current active set is small enough. More formally, assume we performed the $j + 1$ minor minimizations and let $T_{k,j}$ be the tangent space at $x_{k,j}$ with respect to the active bounds in $\mathcal{A}(x_{k,j})$. We stop the minor iteration loop whenever

$$\|P_{T_{k,j}} [\nabla m_k(x_{k,j+1})]\| \leq \kappa_{\text{mlt}} \|P_{T_{k,j}} [\nabla m_k(x_k)]\|,$$

with $\kappa_{\text{mlt}} \in (0, 1)$. This inequality estimates whether relative progress can still be made in the tangent subspace spanned by the free variables.

2.4 Structured Hessian approximation

We now discuss how the Hessian of the AL is iteratively updated when we define the quadratic model of iteration k in Algorithm 2. To set up the context and notation of this paragraph, we wish to approximate the true Hessian

$$\nabla^2 \varphi(x_k) = J_k^\top J_k + \mu C_k^\top C_k + S_k,$$

by a symmetric matrix H_k . We recall that the need for an approximation mainly comes from the fact that evaluating the second-order terms in S_k requires too much time and storage to be done in practice, especially for problems with a large number of variables and residuals.

The first approximation we can think of, and the simplest one, is obtained after merely linearizing the residuals and constraints in expression (2.1), which gives

$$H_k = J_k^\top J_k + \mu C_k^\top C_k. \tag{2.22}$$

The GN approximation (2.22) – in reference to its unconstrained counterpart – is computationally cheap as it involves first-order derivative quantities already required to evaluate the gradient. This matrix is always positive semidefinite. It is positive definite – which guarantees the convexity of the quadratic subproblems (2.13) – if and only if the stacked matrix $\begin{pmatrix} J_k \\ \sqrt{\mu} C_k \end{pmatrix}$ has full column rank, that is, $\ker J_k \cap \ker C_k = \{0\}$. However, it is known to exhibit a slower rate of convergence on problems with nonzero residuals at the solution. This downside is amplified when we are to approximate the Hessian of the Lagrangian, or the AL in our case. Indeed, setting S_k to the zero matrix not only neglects the contribution of the residuals to the curvature of the model, but also neglects the contribution of the Lagrange multipliers, for which there is no a priori reason to vanish.

We thus need to take into account the full Hessian to form an approximation. Looking at the literature on this subject, one can observe that there is a variety of approaches focusing on the unconstrained case [4, 26, 36, 42, 63, 64], as it is a major challenge in improving the efficiency of algorithms for problems with nonzero residuals at the solution. Nevertheless, relevant parallels can be drawn with the AL situation, or the constrained case in general, since these studies explore techniques to deal with second-order terms. Because the first-order terms are readily available and provide curvature information, only the second-order components need to be approximated. Therefore, we look for an approximation of the form

$$H_k = B_k^{\text{GN}} + B_k, \tag{2.23}$$

where B_k^{GN} is the right-hand side of (2.22) and B_k is a symmetric approximation of S_k .

Structured approximations of the AL Hessian have been studied for general objective functions [59] but our approach is closer to the one employed in [39]. In the latter, the authors base their approximation on a secant equation derived from a heuristic initially introduced in the unconstrained case [4] that they adapted to approximate the Hessian of the Lagrangian and use it in an SQP algorithm for constrained nonlinear least-squares. This strategy is also at the heart of the SQN method from the popular package for unconstrained nonlinear least-squares NL2SOL [26, 37]. The idea is the following. If we were to approximate each matrix term of the sum (2.4), we would have

$$B_{k+1} = \sum_{i=1}^{n_r} r_i(x_{k+1}) B_{k+1}^{r_i} + \sum_{i=1}^{n_c} \bar{\lambda}_i(x_{k+1}, \lambda, \mu) B_{k+1}^{c_i}, \quad (2.24)$$

with $B_{k+1}^{r_i} \approx \nabla^2 r_i(x_{k+1})$ and $B_{k+1}^{c_i} \approx \nabla^2 c_i(x_{k+1})$. To get an accurate approximation, given a step s_k , it is reasonable to require

$$B_{k+1}^{r_i} s_k = \nabla r_i(x_{k+1}) - \nabla r_i(x_k) \quad \text{and} \quad B_{k+1}^{c_i} s_k = \nabla c_i(x_{k+1}) - \nabla c_i(x_k), \quad (2.25)$$

i.e., that each Hessian maps the change in the variables to the change in the gradients. Noticing that, for each i , the vectors $\nabla r_i(x_{k+1}) - \nabla r_i(x_k)$ and $\nabla c_i(x_{k+1}) - \nabla c_i(x_k)$ are respectively the i -th rows of $(J_{k+1} - J_k)^\top$ and $(C_{k+1} - C_k)^\top$, summing over the residuals and constraints indices all the terms of (2.25) gives, by (2.24), the structured secant equation

$$B_{k+1} s_k = (J_{k+1} - J_k)^\top r_{k+1} + (C_{k+1} - C_k)^\top \bar{\lambda}_{k+1}. \quad (2.26)$$

To follow the conventional notation of quasi-Newton methods, we will denote the right-hand side of (2.26) by $\tilde{y}_k$ to insist on its distinction with the standard term $y_k := g_{k+1} - g_k$. Note that, as in SQN methods for unconstrained least-squares [42], one has

$$B_k^{\text{GN}} s_k + \tilde{y}_k = y_k,$$

so the resulting approximation satisfies the standard secant equation $H_k s_k = y_k$.

The closest existing proposal from the approximation (2.26) is the one in [39]. However, the two works differ in the algorithmic framework within which the approximation is used. In [39], the authors operate within an SQP method, so their approximation targets the Hessian of the Lagrangian and is used to build a quadratic subproblem in which the constraints appear linearized, whereas ours targets the Hessian of the AL and is used within a minimization over a fixed polyhedron. In particular, the curvature contributed by the nonlinear constraints enters our secant equation through the shifted multipliers $\bar{\lambda}_{k+1}$, which are fixed during the inner minimization, and not through Lagrange multiplier estimates updated after every iteration.

We can now derive an update formula from equation (2.26). In our implementation, we apply the SR1 update

$$B_{k+1} = B_k + \frac{(\tilde{y}_k - B_k s_k)(\tilde{y}_k - B_k s_k)^\top}{\langle \tilde{y}_k - B_k s_k, s_k \rangle}, \quad (2.27)$$

if $\langle \tilde{y}_k - B_k s_k, s_k \rangle \neq 0$. When the denominator in (2.27) is zero, we simply set $B_{k+1} = B_k$. To avoid numerical errors, we apply the update when

$$|\langle \tilde{y}_k - B_k s_k, s_k \rangle| \geq \kappa_{\text{sds}} \|s_k\| \|\tilde{y}_k - B_k s_k\|, \quad (2.28)$$

for $\kappa_{\text{sds}} \in (0, 1)$. Inequality (2.28) is one of the most common safeguards used in SR1 methods [48, Section 6.2].

The choice of the SR1 update formula is motivated by its robustness in the least-squares context, as the exhaustive benchmarks in [42] show, and its tendency to better approximate the true Hessian with each iteration [18]. The approximation could thus be indefinite, which could be considered as a weakness compared to BFGS or DFP updates, which are guaranteed to be positive definite as long as this is the case for the initial approximation B_0 . However, as highlighted in Algorithm 3, the use of the conjugate gradient method in a trust-region framework handles, and even exploits, the potential non-convexity of the subproblems. Moreover, this update does not require a curvature condition $\langle s_k, y_k \rangle > 0$ to be satisfied. In other works on the constrained case, SQN methods are used to approximate the projected Hessian of the Lagrangian [60] – or of a merit function [43] – in the null space of the active constraints. The BFGS update is then preferred because this matrix is, under standard assumptions, positive semidefinite, according to the second-order necessary conditions.

We end our description of the Hessian approximation by discussing hybrid updates, a feature commonly used in SQN methods for nonlinear least-squares, either in the unconstrained [1, 26, 28] or constrained [39, 60] case. The idea is to test at every iteration whether the problem has small or large residuals at the solution. In the latter case, a structured update is used to increase the accuracy of the approximated Hessian whereas in the former, the GN approximation is employed, as it is more likely to correspond to the true Hessian. Note that the update, such as (2.27), is still computed at every iteration to continue accumulating curvature information. We use a strategy inspired by [28] adapted to the constrained case in [60] that is based on the asymptotic behavior of the ratio

$$\zeta_k = \frac{\varphi(x_k) - \varphi(x_{k+1})}{\varphi(x_k)}.$$

The approximated Hessian is then updated following the switching rule

$$H_{k+1} = \begin{cases} B_{k+1}^{\text{GN}} + B_{k+1} & \text{if } \zeta_k \leq \kappa_{\text{hyb}} \\ B_{k+1}^{\text{GN}} & \text{otherwise,} \end{cases} \quad (2.29)$$

where κ_{hyb} is a parameter in $(0, 1)$. In [60], this update is also tested with the value of the AL where it plays the role of a merit function. As we will see in Section 4, it has an important impact on the performance of the algorithm.

2.5 Summary and implementation details

We end our discussion by summarizing the relations between outer, inner and minor iterates and the default values for the parameters of the algorithm presented in this section. Recall that the outer iterates are indexed with K and the inner iterates with k . The outer iterates x_K are the main iterates of the algorithm and are approximate minimizers of the AL. Each outer iterate is formed after a sequence of inner iterates $(x_k)_k$, linked by $x_{k+1} = x_k + s_k$, where s_k is an approximate solution of the QP (2.13). In turn, each inner iterate is formed after a sequence of M minor iterates $x_{k,j}$, linked by $x_{k,j+1} = x_{k,j} + w_{k,j}$, where $w_{k,j}$ is a descent direction obtained by applying the projected conjugate gradient Algorithm 3. The relations between the three levels of iterations are illustrated in Figure 1.

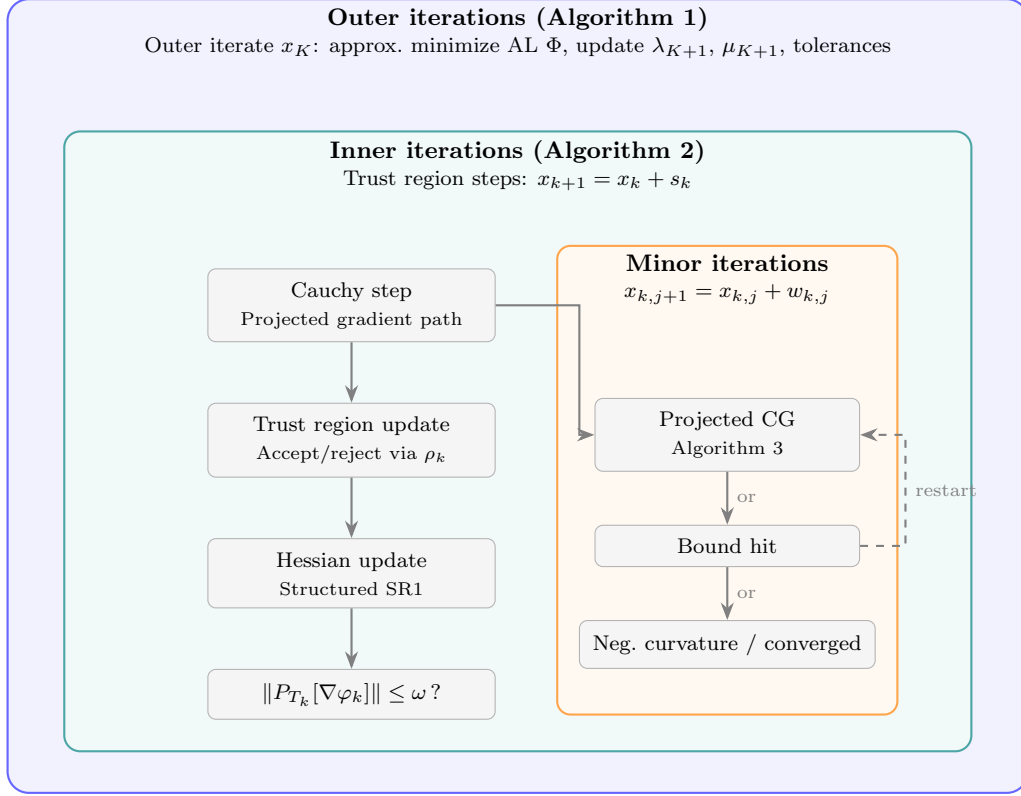


Figure 1: Hierarchy of the three iteration levels in the AL algorithm. The outer loop updates the Lagrange multipliers and penalty parameter. Each outer iteration triggers a sequence of trust-region inner iterations, and each inner iteration computes its step via a Cauchy point followed by minor iterations using the projected conjugate gradient method.

Computing projections From our description of the step computation, we can see that whether during the Cauchy step calculation phase or during the minor iteration loop, the active set – and thus the projection operator $\tilde{P}$ at (2.9) – changes over the course of an iteration. It is therefore worth examining how this operator is updated. One can compute the projection of a vector v , i.e., $\tilde{P}v$, by first solving, for an auxiliary variable y , the linear system

$$\left(\tilde{A}\tilde{A}^\top\right)y = \tilde{A}v, \quad (2.30)$$

and then retrieving the projection as

$$v - \tilde{A}^\top y. \quad (2.31)$$

This approach, referred to as the *normal equations* approach [32], requires solving the system (2.30). This is done by using the Cholesky decomposition of $\tilde{A}\tilde{A}^\top = LL^\top$ – with L lower triangular – and system (2.30) is thus reduced to two successive lower triangular systems. The latter factorization is well defined since the matrix $\tilde{A}\tilde{A}^\top$ is symmetric by construction and positive definite by Assumption 4. Moreover, the block structure of $\tilde{A}\tilde{A}^\top$ can be exploited to incrementally maintain the L factor – instead of forming it from scratch – across the changes

in the active set. Given an active set $\mathcal{A} \subset \{1, \dots, n\}$ of cardinality $p < n - m$, and writing $A_{\mathcal{A}} := AE_{\mathcal{A}}$ for the restriction of A to the columns indexed by $\mathcal{A}$, $\tilde{A}\tilde{A}^\top$ can be written

$$\begin{bmatrix} AA^\top & A_{\mathcal{A}} \\ A_{\mathcal{A}}^\top & I_p \end{bmatrix}. \quad (2.32)$$

From (2.32), one can show [30, see Section 4.2.9] that the L factor has the corresponding block pattern

$$L = \begin{bmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{bmatrix},$$

where L_{11} is the $m \times m$ Cholesky factor of $AA^\top$, $L_{21} := (L_{11}^{-1}A_{\mathcal{A}})^\top$ is of size $p \times m$, and L_{22} is the $p \times p$ Cholesky factor of $G := I_p - L_{21}L_{21}^\top$. The matrix A being fixed, forming its Cholesky decomposition and $L_{11}^{-1}A$ just has to be done once at the start of the algorithm. Therefore, L_{11} and L_{21} – the latter given by extracting the columns of $L_{11}^{-1}A$ indexed by $\mathcal{A}$ – are both available quantities and only the L_{22} part needs to be maintained. Since adding an index to $\mathcal{A}$ results in adding a column to $A_{\mathcal{A}}$, and thus in appending a row and a column to G , the new factor L_{22} can be updated incrementally by applying a bordered update to the existing one in $\mathcal{O}(p^2)$ flops, which is less costly than the $\mathcal{O}((m+p)^3)$ time complexity required to form the entire L factor from scratch.

Bound constrained problems Our intention with the work presented in this paper was to design an algorithm able to directly handle linear constraints of the form (1.2) but we also accept problems where linear constraints are only bounds and then

$$\Omega = \{x \in \mathbb{R}^n \mid l \leq x \leq u\}.$$

The framework we have presented remains valid for this formulation. The only practical difference is in the computation of projections. In the bound-constrained case, the projection of a vector x on the set Ω has components

$$(P_\Omega[x])_i = \begin{cases} l_i & \text{if } x_i < l_i \\ x_i & \text{if } x_i \in [l_i, u_i] \\ u_i & \text{if } x_i > u_i \end{cases} \quad (2.33)$$

In our implementation, the only practical difference with the polyhedral case is that we can directly use the norm of the projected gradient

$$\|x_K - P_\Omega[x_K - \nabla_x \Phi_K]\|,$$

as a criticality measure. Note that the projections onto tangent spaces are also simplified, as they are now given by setting the components corresponding to active bounds to 0.

Parameter conditions Algorithm 1 requires $\mu_0 > 0$, $\tau > 1$, positive tolerance constants $\omega, \eta, \kappa_\omega, \kappa_\eta, \beta_\omega, \beta_\eta$ and positive stopping tolerances ω_*, η_* , the tolerance sequences being updated following [22, Chapter 14]. The constants driving the inner minimization of Algorithm 2 are tied to the subproblem criticality tolerance, so that at outer iteration K we set $\kappa_{cg} = \kappa_{mlt} = \omega_K$, while the safeguard constant $\kappa_{sds} \in (0, 1)$ of (2.28) is taken as the square root of the relative machine precision.

At the start of each outer iteration, the initial approximation of the second-order terms of the Hessian is $B_0 = 0$, and the hybrid switching scheme (2.29) is monitored through a constant $\kappa_{\text{hyb}} \in (0, 1)$, as in [60].

We now discuss the trust-region handling. The initial radius value for Algorithm 2 is set to

$$\Delta_0 = \delta_0 \|g_0\|_\infty, \quad \delta_0 > 0.$$

The mechanism to update the trust region given in (2.16) still leaves important flexibility. We choose to follow a standard strategy similar to the one presented in Chapter 17 of [22]. We also added a refinement to handle the case where the ratio ρ_k is negative, which can happen when there is very poor agreement between the function and the model. In such cases, we reduce the radius more severely by taking

$$\Delta_{k+1} = \gamma_1 \Delta_k.$$

The complete update rule for the trust-region radius is then

$$\Delta_{k+1} = \begin{cases} \max(\alpha_2 \|s_k\|_\infty, \Delta_k) & \text{if } \rho_k \geq \eta_2 \\ \Delta_k & \text{if } \rho_k \in [\eta_1, \eta_2) \\ \alpha_1 \|s_k\|_\infty & \text{if } \rho_k \in [0, \eta_1) \\ \min(\alpha_1 \|s_k\|_\infty, \gamma_1 \Delta_k) & \text{if } \rho_k < 0, \end{cases}$$

whose constants satisfy $0 < \alpha_1 < 1 < \alpha_2$, $0 < \eta_1 \leq \eta_2 < 1$ and $0 < \gamma_1 < 1$, refining the generic scheme (2.16). The specific numerical values of all parameters used in our tests are reported in Section 4.

3 Convergence analysis

In this section, we study the convergence of Algorithm 1 towards a first-order critical point of problem (1.1). We start by showing global convergence of Algorithm 2, implying that the inner minimization phase of Algorithm 1 is always well defined. We then establish global convergence of the main algorithm. Our analysis is based on the criticality measure, i.e., the norm of the projected gradient, which differs from the reduced gradient norm (2.10) used in our implementation. At the end of this section, we discuss the validity of our convergence results when (2.10) is used as an inner iteration termination condition instead of (2.6).

3.1 Global convergence of the inner minimization

The analysis below is carried out over the polyhedron (1.2), but because our inner-minimization procedure closely resembles the SBMIN algorithm [16] – used in the bound-constrained inner-minimization phase of LANCELOT [19] – the architecture of the arguments follows the global convergence proof in [15]. Compared to the bound-constrained case, the projection onto Ω has no closed form, so the active-set arguments must be re-established and the intermediate results re-derived in the polyhedral setting. More precisely, the polyhedral structure is confined to the first half of the analysis. Lemmas 1 to 4 and the guaranteed decrease inequality (3.20) are the results that have to be established anew, because the bound-constrained arguments they replace all rest on the projection being computable componentwise. Once that inequality is available, the polyhedron plays no further role and the standard convergence theory of trust region methods applies unchanged.

We first make the standard assumption

Assumption 5. *The set $\mathcal{X} = \{x \mid \varphi(x) \leq \varphi(x_0)\} \cap \Omega$ is non-empty and compact.*

By Assumption 1, it is implicit that the function φ is twice continuously differentiable on $\mathcal{X}$ so we will not state this in the propositions given in this section.

We recall that the quadratic model is of the form

$$q_k(s) = \frac{1}{2} \langle s, H_k s \rangle + \langle g_k, s \rangle,$$

where $g_k = \nabla \varphi(x_k)$ and H_k is an approximation of the Hessian based on the structured SR1 formula from Section 2.4. We formulate two assumptions relative to those approximations. The norm used is the induced norm on matrices, i.e., $\|M\| := \sup_{\|x\|=1} \|Mx\|$ for a given matrix M .

Assumption 6. *Defining, for every iteration index k , the scalars b_k by*

$$b_k = 1 + \max_{0 \leq i \leq k} \|H_i\|,$$

we require that the series $\sum_k \frac{1}{b_k}$ diverges to ∞ .

The second assumption states that the norm of the approximating Hessians should not increase too fast compared with the speed of convergence of the function values.

Assumption 7.

$$\lim_{k \rightarrow \infty} b_k (\varphi_{k+1} - \varphi_k) = 0.$$

The projected gradient path is defined as

$$s_k(t) = P_\Omega [x_k - t g_k] - x_k \text{ for } t \geq 0.$$

The reduction of the model along the projected gradient path may thus be defined as the piecewise quadratic function

$$\psi(t) = q_k(s_k(t)),$$

and we denote by t_k^C the first local minimum of ψ subject to the trust-region constraint

$$\|s_k(t)\|_\infty \leq \Delta_k.$$

The associated Cauchy step is

$$s_k^C = s_k(t_k^C).$$

Because of the choice of the ℓ_∞ norm, computing $s_k(t)$ within the trust region corresponds to projecting the direction $x_k - t g_k$ onto

$$\{d \in \mathbb{R}^n \mid A d = 0, l_k \leq d \leq u_k\}, \quad (3.1)$$

with $(l_k)_i = \max(-\Delta_k, l_i - (x_k)_i)$ and $(u_k)_i = \min(\Delta_k, u_i - (x_k)_i)$ for all $i = 1, \dots, n$.

We assume that the total step s_k produces a fraction of the reduction achieved by the Cauchy point, in the sense that

$$q_k(s_k) \leq \kappa_{\text{fcd}} q_k(s_k^C), \quad (3.2)$$

for $\kappa_{\text{fcd}} \in (0, 1]$.

We detail the behavior of the polygonal line $s_k(t)$. Let $\mathcal{A}(t)$ be the set of bounds l_k or u_k satisfied with equality at $s_k(t)$. Notice that this definition slightly differs from the active set $\mathcal{A}(x)$ defined earlier, as the new formulation now takes into account the trust region. At first, if no bounds are active at x_k , projecting on the set (3.1) for t close to 0 is equivalent to projecting onto the null space of A . As t increases, the direction might hit several bounds. Since the set of active bounds can only be increased, we have that

$$\mathcal{A}(t) \subseteq \mathcal{A}(t') \quad \text{for all } 0 < t \leq t'. \quad (3.3)$$

Let $0 = t_0 < t_1 < \dots < t_p$ be the successive values of t at which the projected step $s_k(t)$ hits a bound, also called breakpoints. Hitting a bound not only affects the corresponding components, which are now fixed, but it also affects the other components, as the steepest direction must now be projected on a subspace

$$\{d \in \mathbb{R}^n \mid Ad = 0, d_i = 0 \text{ for all fixed components } i\}.$$

This motivates adapting the notion of tangent space to a point on the projected gradient path

$$T(t) := \{d \in \mathbb{R}^n \mid Ad = 0, d_i = 0 \text{ for all fixed components } i \in \mathcal{A}(t)\},$$

for $t \geq 0$. This enables us to give a recursive expression of $s_k(t)$ on each interval $[t_i, t_{i+1})$, with $0 \leq i \leq p$, as

$$s_k(t) = (t - t_i)P_{T(t_i)}[-g_k] + s_k(t_i). \quad (3.4)$$

We also define the reduced gradient on the projected gradient path

$$z_k(t) := P_{T(t)}[g_k]. \quad (3.5)$$

Note that the reduced gradient, and hence the path $s_k(t)$, are well defined because of the full-rank Assumption 2 on the matrix A . As for the differentiability of φ , we will not restate it in the results of this section.

We now state a first lemma on how the tangent spaces and the reduced gradients at two different positions compare to each other. This result has no counterpart in the bound-constrained setting. There, the projection is computed componentwise, each component of the projected path is an explicit monotone piecewise linear function of t , and the monotonicity of the active set follows immediately. Over Ω , the components of the projected step are coupled by the linear equalities, so the nesting of the active sets has to be obtained from the geometry of the projection itself.

Lemma 1. *For all $0 < t < t'$, we have that*

$$T(t') \subseteq T(t),$$

and

$$\|z_k(t')\| \leq \|z_k(t)\|. \quad (3.6)$$

Proof. The first statement follows from the inclusion (3.3). Inequality (3.6) then results from the fact that projecting the same vector on a smaller linear subspace reduces the norm of its projection. $\square$

When referring to the tangent space and the reduced gradient at a given breakpoint t_i , we will make use of the shorthand notation $T_i := T(t_i)$ and $z_i := z_k(t_i)$. Notice that the active

set is fixed on each interval $[t_i, t_{i+1})$, hence so are the tangent space and the reduced gradient. Also, we can deduce from Lemma 1 that the tangent spaces associated with each breakpoint form a finite sequence of nested linear subspaces

$$T_0 \supseteq T_1 \supseteq \dots \supseteq T_p.$$

We can now expand equation (3.4) into

$$s_k(t) = -(t - t_i)z_i - \sum_{j=0}^{i-1} (t_{j+1} - t_j)z_j, \quad (3.7)$$

which shows that on each interval (t_i, t_{i+1}) , $s_k(t)$ is differentiable with respect to t and that

$$s'_k(t) = -z_k(t) = -z_i, \quad (3.8)$$

for $t \in (t_i, t_{i+1})$.

The properties established so far only describe how the projected gradient path behaves over a polyhedron and do not involve the objective function or the quadratic model. The next proposition isolates these results, as they apply to any method whose step computation relies on a Cauchy point over a set of the form (1.2).

Proposition 2. *Let $\Omega = \{x \in \mathbb{R}^n \mid Ax = b, l \leq x \leq u\}$ be a non-empty polyhedron with A of full row rank, let $\bar{x} \in \Omega$ and let $g \in \mathbb{R}^n$. Define the projected gradient path $s(t) = P_\Omega[\bar{x} - tg] - \bar{x}$ for $t \geq 0$, the tangent space $T(t)$ associated with the bounds active at $s(t)$, and the reduced gradient $z(t) = P_{T(t)}[g]$. Then the following hold.*

- (i) *The path is piecewise linear. There are finitely many breakpoints $0 = t_0 < t_1 < \dots < t_p$ outside of which s is differentiable, with $s'(t) = -z(t)$, and z is constant on each interval $[t_i, t_{i+1})$.*
- (ii) *The active sets are nested along the path, and so are the tangent spaces and the reduced gradient norms: for all $0 < t \leq t'$, $\mathcal{A}(t) \subseteq \mathcal{A}(t')$, $T(t') \subseteq T(t)$ and $\|z(t')\| \leq \|z(t)\|$.*
- (iii) *The path is recovered from its reduced gradients by*

$$s(t) = - \int_0^t z(\tau) d\tau, \quad (3.9)$$

the integral being understood as the sum of the integrals over the segments delimited by the breakpoints.

Proof. Statement (i) follows from (3.4) and (3.7)–(3.8), the number of breakpoints being finite because, by (3.3), a bound that becomes active stays active. Statement (ii) is (3.3) together with Lemma 1. Statement (iii) is obtained by integrating s' over each segment $[t_i, t_{i+1}]$ and summing, the one-sided derivatives of s at the breakpoints being well defined and equal to $-z_i$ by (3.8). $\square$

In the bound-constrained case, where the projection is computed componentwise and each component of $s(t)$ is an explicit piecewise linear function of t , these properties are immediate and are used without comment. Over a general polyhedron the components are coupled by the linear equalities, and (3.9) is what replaces the componentwise description in every argument

that follows. Proposition 2 and the Cauchy point procedure of Appendix A, which computes the first local minimizer of a quadratic along that path, are independent of the augmented Lagrangian context in which we use them.

The bound-constrained proof of the result below exploits the separability of the projected path, each component being known in closed form. That separability is lost over the polyhedron, and we replace it by the integral representation of the path through its reduced gradients, which follows from (3.8) and is controlled by Lemma 1. We start by establishing an inequality on the decrease of the objective function after taking step s_k . This will involve the norm of the projected gradient,

$$h_k := \|s_k(1)\|. \quad (3.10)$$

Lemma 3. *If Assumption 5 holds and $h_k > 0$, then*

$$\|z_k(t_k^{(1)})\| \geq \frac{h_k}{2},$$

where

$$t_k^{(1)} = \frac{h_k}{2\kappa_{\text{ubg}}},$$

and κ_{ubg} is the constant defined by $\kappa_{\text{ubg}} := \max(1, \max_{x \in \mathcal{X}} \|\nabla \varphi(x)\|)$.

Proof. First note that the constant κ_{ubg} is well defined since φ is continuously differentiable on $\mathcal{X}$, the latter being compact by Assumption 5.

Since the set Ω is closed and convex, the projection mapping $P_\Omega[\cdot]$ is non-expansive, so that for any $t \geq 0$

$$\begin{aligned} \|s_k(t)\| &= \|P_\Omega[x_k - tg_k] - x_k\| \\ &= \|P_\Omega[x_k - tg_k] - P_\Omega[x_k]\| \\ &\leq \|x_k - tg_k - x_k\| \\ &\leq t\|g_k\|. \end{aligned}$$

Choosing $t = t_k^{(1)}$ and because $\|g_k\| \leq \kappa_{\text{ubg}}$ by definition of κ_{ubg} , we get

$$\|s_k(t_k^{(1)})\| \leq \frac{h_k}{2}. \quad (3.11)$$

Now, define $t_k^{(2)}$ as the smallest $t \geq 0$ such that

$$\|s_k(t_k^{(2)})\| = h_k. \quad (3.12)$$

By (3.11), we have

$$0 < t_k^{(1)} < t_k^{(2)} \leq 1. \quad (3.13)$$

By Proposition 2(iii) applied with $\bar{x} = x_k$ and $g = g_k$, the path admits the integral representation

$$s_k(t) = - \int_0^t z_k(\tau) d\tau. \quad (3.14)$$

Now let i_1 and i_2 be the breakpoint indices such that $t_k^{(1)} \in [t_{i_1}, t_{i_1+1})$ and $t_k^{(2)} \in [t_{i_2}, t_{i_2+1})$ respectively. Then, by (3.14),

$$s_k(t_k^{(2)}) - s_k(t_k^{(1)}) = - \int_{t_k^{(1)}}^{t_k^{(2)}} z_k(t) dt,$$

which leads to

$$\begin{aligned} \|s_k(t_k^{(2)}) - s_k(t_k^{(1)})\| &\leq \int_{t_k^{(1)}}^{t_k^{(2)}} \|z_k(t)\| dt \\ &\leq \int_{t_k^{(1)}}^{t_k^{(2)}} \|z_k(t_k^{(1)})\| dt \\ &\leq (t_k^{(2)} - t_k^{(1)}) \|z_k(t_k^{(1)})\|, \end{aligned}$$

where we have used (3.6) to bound the norm of the reduced gradient on $[t_k^{(1)}, t_k^{(2)}]$. Combined with (3.13) and (3.12), we get

$$\begin{aligned} \frac{h_k}{2} = \|s_k(t_k^{(2)})\| - \frac{h_k}{2} &\leq \|s_k(t_k^{(2)})\| - \|s_k(t_k^{(1)})\| \\ &\leq \|s_k(t_k^{(2)}) - s_k(t_k^{(1)})\| \\ &\leq (t_k^{(2)} - t_k^{(1)}) \|z_k(t_k^{(1)})\| \\ &\leq \|z_k(t_k^{(1)})\|, \end{aligned}$$

which is the desired inequality. $\square$

The next lemma gives an upper bound on the quadratic model $\psi(t)$ in an interval of interest. Its proof is again conducted along the breakpoints of the polygonal path rather than componentwise.

Lemma 4. *If Assumption 5 holds and for some $t_k^{(3)} > 0$, one has*

$$\alpha_k = \|z_k(t_k^{(3)})\| > 0,$$

then, if T is the set of points in $[0, t_k^{(3)}]$ at which the piecewise quadratic ψ is differentiable,

$$\psi'(t) \leq -\alpha_k^2 + t_k^{(3)} \kappa_{\text{ubg}}^2 \|H_k\|, \quad (3.15)$$

for all $t \in T$.

Furthermore,

$$\psi'(t) \leq -\frac{1}{2} \alpha_k^2,$$

for all $t \in T \cap [0, t_k^{(4)}]$ and

$$\psi(t) \leq -\frac{\alpha_k^2}{2} t,$$

for all $t \in [0, t_k^{(4)}]$ where

$$t_k^{(4)} = \min \left(t_k^{(3)}, \frac{\alpha_k^2}{2\kappa_{\text{ubg}}^2 (\|H_k\| + 1)} \right). \quad (3.16)$$

Proof. For $t \in T$, let i be the breakpoint index such that $t \in (t_i, t_{i+1})$. On the latter interval, ψ is differentiable and we have, by expression (3.7),

$$\begin{aligned}\psi'(t) &= \langle g_k, s'_k(t) \rangle + \langle s_k(t), H_k s'_k(t) \rangle \\ &= -\langle g_k, z_i \rangle - \langle s_k(t), H_k z_i \rangle.\end{aligned}$$

Because z_i is, by definition, the orthogonal projection of the vector g_k on a linear subspace, we have

$$\begin{aligned}\langle g_k, z_i \rangle &= \langle z_i, z_i \rangle \\ &\geq \|z_k(t_k^{(3)})\|^2 = \alpha_k^2,\end{aligned}\tag{3.17}$$

where the last inequality follows from the monotonicity of the reduced gradient norm on $[0, \infty)$.

We now look at the quadratic terms. First, by the Cauchy–Schwarz inequality,

$$|\langle s_k(t), H_k z_i \rangle| \leq \|s_k(t)\| \|H_k z_i\|.$$

Then, applying the triangle inequality to expression (3.7), we obtain

$$\begin{aligned}\|s_k(t)\| &\leq (t - t_i) \|z_i\| + \sum_{j=0}^{i-1} (t_{j+1} - t_j) \|z_j\| \\ &\leq (t - t_i) \|g_k\| + \sum_{j=0}^{i-1} (t_{j+1} - t_j) \|g_k\| \\ &\leq t \|g_k\| \leq t_k^{(3)} \kappa_{\text{ubg}},\end{aligned}\tag{3.18}$$

where we have used the facts that for all breakpoint indices j , $\|z_j\| \leq \|g_k\| \leq \kappa_{\text{ubg}}$ and that the scalar t is taken in T . For the remaining terms,

$$\|H_k z_i\| \leq \|H_k\| \|z_i\| \leq \kappa_{\text{ubg}} \|H_k\|.$$

Combining the latter inequality with (3.18) bounds the quadratic terms by

$$|\langle s_k(t), H_k z_i \rangle| \leq t_k^{(3)} \kappa_{\text{ubg}}^2 \|H_k\|.\tag{3.19}$$

Hence, using (3.17) and (3.19),

$$\psi'(t) \leq -\alpha_k^2 + t_k^{(3)} \kappa_{\text{ubg}}^2 \|H_k\|,$$

for all $t \in T$, which proves (3.15). Now, considering $t_k^{(4)}$ as defined by (3.16), since $t_k^{(4)} \leq t_k^{(3)}$, we get that $\|z_k(t_k^{(4)})\| \geq \alpha_k > 0$. The above reasoning based on $t_k^{(4)}$ thus yields

$$\psi'(t) \leq -\alpha_k^2 + t_k^{(4)} \kappa_{\text{ubg}}^2 \|H_k\| \leq -\frac{\alpha_k^2}{2},$$

for $t \in T$, $t \leq t_k^{(4)}$. It follows that, for all $t \in [0, t_k^{(4)}]$,

$$\psi(t) \leq -\frac{\alpha_k^2}{2} t,$$

which completes the proof. $\square$

We can now establish a bound on the decrease guaranteed by the step. This is the last result of the section in which the polyhedral structure intervenes, and the point at which our development rejoins the bound-constrained theory.

Theorem 5. *If Assumptions 5 and 6 hold and $h_k > 0$, then*

$$q_k(s_k) \leq -\kappa_{\text{mdc}} h_k^2 \min\left(\frac{h_k^2}{b_k}, \Delta_k\right), \quad (3.20)$$

where

$$\kappa_{\text{mdc}} = \frac{\kappa_{\text{fcd}}}{64\kappa_{\text{ubg}}^2}.$$

Furthermore, if iteration k is successful, then

$$\varphi(x_k) - \varphi(x_k + s_k) \geq \kappa_{\text{sdc}} h_k^2 \min\left(\frac{h_k^2}{b_k}, \Delta_k\right), \quad (3.21)$$

with $\kappa_{\text{sdc}} = \eta_1 \kappa_{\text{mdc}}$.

Proof. We first observe that if we use $t_k^{(1)}$ as $t_k^{(3)}$ in the proof of Lemma 4 and apply Lemma 3, we get

$$\psi(t) \leq -\frac{h_k^2}{8}t, \quad (3.22)$$

for $t \in [0, t_k^{(5)}]$ with

$$t_k^{(5)} := \min\left(t_k^{(1)}, \frac{h_k^2}{8\kappa_{\text{ubg}}^2(\|H_k\| + 1)}\right) = \frac{h_k^2}{8\kappa_{\text{ubg}}^2(\|H_k\| + 1)}.$$

We will bound the decrease obtained with the step depending on where the point associated with $t_k^{(5)}$ lies with respect to the trust-region boundary.

First, assume that $\|s_k(t_k^{(5)})\|_\infty \leq \Delta_k$. By (3.22) and Lemma 4, one has $\psi'(t) < -h_k^2/8 < 0$ for all $t \in [0, t_k^{(5)}]$, so ψ is strictly decreasing on this segment. Because the Cauchy point is the first local minimizer of ψ subject to the trust region and the trust region is not violated at $t_k^{(5)}$, we must have $t_k^C \geq t_k^{(5)}$. Therefore,

$$\psi(t_k^C) \leq \psi(t_k^{(5)}) \leq -\frac{h_k^2}{8}t_k^{(5)}.$$

Then, by (3.2), $q_k(s_k) \leq \kappa_{\text{fcd}} q_k(s_k^C)$ and it follows that

$$q_k(s_k) \leq -\kappa_{\text{fcd}} \frac{h_k^2}{8} t_k^{(5)}. \quad (3.23)$$

Now, assume that $\|s_k(t_k^{(5)})\|_\infty > \Delta_k$. The latter implies that $\|s_k(t_k^C)\|_\infty = \Delta_k$ and from

$$\left\|s_k\left(\frac{\Delta_k}{\kappa_{\text{ubg}}}\right)\right\|_\infty \leq \left\|s_k\left(\frac{\Delta_k}{\kappa_{\text{ubg}}}\right)\right\| \leq \frac{\Delta_k}{\kappa_{\text{ubg}}} \|g_k\| \leq \Delta_k,$$

we can deduce that

$$t_k^C \geq \frac{\Delta_k}{\kappa_{\text{ubg}}}.$$

Therefore, using (3.22) with $t = t_k^C$ and (3.2) implies

$$q_k(s_k) \leq -\kappa_{\text{fcd}} \frac{h_k^2}{8\kappa_{\text{ubg}}} \Delta_k \leq -\kappa_{\text{fcd}} \frac{h_k^2}{64\kappa_{\text{ubg}}^2} \Delta_k, \quad (3.24)$$

because $\kappa_{\text{ubg}} \geq 1$. The inequality (3.20) results from gathering (3.23), (3.24) and the definition of scalar b_k .

Finally, if the step is successful, we get inequality (3.21) by using (3.20) and the step acceptance condition $\rho_k > \eta_1$ from Algorithm 2. $\square$

Once we have established the guaranteed decrease inequality (3.20), the rest of the proof does not involve the polyhedral structure of the constraints and we can fall back on the standard convergence theory of trust-region methods. We state the main convergence theorem, whose proof and intermediate developments can be found in [15].

Theorem 6 ([15] Theorem 11). *If Assumptions 5, 6 and 7 hold, then*

$$\lim_{k \rightarrow \infty} h_k = 0.$$

3.2 Global convergence of the algorithm

We now establish that every limit point of the outer iteration sequence is a first-order critical point of (1.1), for a feasible set combining nonlinear equalities with a general polyhedron. The argument follows the structure of the global convergence proof outlined in [21], where the linear constraints are inequalities only; the constraint qualification is thus restated to account for the linear equalities present alongside the bounds. We first make the following assumption about the iterates considered.

Assumption 8. *The iterates produced by Algorithm 1 lie within a closed bounded domain of $\mathbb{R}^n$.*

Let $(x_K)_K$ be a sequence of such iterates. Assumption 8 then ensures the existence of a subsequence indexed by $\mathcal{K} \subseteq \mathbb{N}$ converging to a limit point x_* . Since the iterates satisfy $x_K \in \Omega$ for all K , we also have $x_* \in \Omega$. We recall from (2.7) that $\mathcal{A}(x)$ is the set of bounds active at x and $T(x)$ the associated tangent space, and we abbreviate $\mathcal{A}_* := \mathcal{A}(x_*)$. We denote by $\tilde{A}_*$ the matrix gathering the linear constraints active at x_* ,

$$\tilde{A}_* := \begin{pmatrix} A \\ E_*^\top \end{pmatrix},$$

where E_* is the submatrix of the identity whose columns are e_i , $i \in \mathcal{A}_*$. Consistently with the well-posedness of the projector (2.9), $\tilde{A}_*$ has full row rank, so $T(x_*) = \text{null}(\tilde{A}_*)$ and there is an orthonormal matrix N_* whose columns form a basis of $T(x_*)$. The next assumption ensures that the null space of the active linear constraints is large enough to achieve feasibility of the nonlinear constraints, and that the gradients of those constraints are linearly independent within that space.

Assumption 9. *The rank of the matrix $C(x_*)N_*$ is no smaller than n_c at any limit point x_* of the sequence $(x_K)_K$.*

We now introduce the remaining notation and notions used in our convergence analysis. Since Ω is of the form (1.2), the normal cone to Ω at $x \in \Omega$ can be defined as

$$\mathcal{N}_\Omega(x) = \left\{ A^\top \xi + \sum_{i \in \mathcal{A}^+(x)} \sigma_i^+ e_i - \sum_{i \in \mathcal{A}^-(x)} \sigma_i^- e_i \mid \xi \in \mathbb{R}^m, \sigma_i^+, \sigma_i^- \geq 0 \right\}, \quad (3.25)$$

where the equality multipliers ξ are free in sign while the bound multipliers are signed. The first-order criticality condition $P_\Omega[x_* - \nabla_x \mathcal{L}(x_*, \lambda_*)] = x_*$ of problem (1.1) is then equivalent to

$$-\nabla_x \mathcal{L}(x_*, \lambda_*) \in \mathcal{N}_\Omega(x_*).$$

Finally, for vectors x at which the matrix $C(x)N_*N_*^\top C(x)^\top$ is non-singular, the least-squares multipliers associated with $\tilde{A}_*$ are defined by

$$\lambda(x) = -((C(x)N_*)^\dagger)^\top N_*^\top \nabla f(x), \quad (3.26)$$

where $(C(x)N_*)^\dagger = N_*^\top C(x)^\top (C(x)N_*N_*^\top C(x)^\top)^{-1}$ is the right pseudoinverse of $C(x)N_*$. When $(C(x)N_*)^\dagger$ is well defined, Assumption 1 ensures that $\lambda(x)$ is differentiable and that its Jacobian $\nabla \lambda(x)$ is bounded in a neighborhood of x_* (see [21, Lemma 2.1]).

We can now proceed with the global convergence proof, starting with a first lemma giving a bound on the reduced gradient norm. This bound is specific to our setting. When Ω reduces to a box, the comparison between the criticality measure used by the algorithm and the projected gradient on the tangent space at the limit point is obtained componentwise. Here we derive it from the characterization of the projection by the normal cone $\mathcal{N}_\Omega(v_K)$, whose expression (3.25) identifies it with the row space of the matrix (2.8) gathering the active linear constraints.

Lemma 7. *Let $(x_K)_{K \in \mathcal{K}} \in \Omega$ be a sequence converging to x_* and such that*

$$\|P_\Omega[x_K - \nabla_x \Phi_K] - x_K\| \leq \omega_K,$$

with $\lim_{K \rightarrow \infty} \omega_K = 0$. Then

$$\|P_{T(x_*)}[\nabla_x \Phi_K]\| \leq \omega_K,$$

for all $K \in \mathcal{K}$ large enough.

Proof. To lighten the notation, we write $v_K := P_\Omega[x_K - \nabla_x \Phi_K]$ and $d_K := v_K - x_K$. We have

$$\begin{aligned} \|v_K - x_*\| &\leq \|v_K - x_K\| + \|x_K - x_*\| \\ &\leq \omega_K + \|x_K - x_*\|. \end{aligned}$$

Since, by hypothesis, $\omega_K \rightarrow 0$ and $x_K \rightarrow x_*$ for $K \in \mathcal{K}$, the two terms on the right-hand side converge to 0, which implies that we also have

$$\lim_{K \in \mathcal{K}} v_K = x_*.$$

Therefore, there is a threshold index K_0 such that all bounds inactive at x_* are also inactive at v_K for $K \geq K_0$. This is equivalent to $\mathcal{A}(v_K) \subseteq \mathcal{A}_*$ for $K \geq K_0$. We then have the reverse inclusion for the associated tangent spaces $T(x_*) \subseteq T(v_K)$, giving

$$\|P_{T(x_*)} [\nabla_x \Phi_K]\| \leq \|P_{T(v_K)} [\nabla_x \Phi_K]\|, \quad (3.27)$$

whenever $K \in \mathcal{K}$ is above the threshold index K_0 .

Recall from Section 2 that $T(v_K)$ is the null space of a matrix $\tilde{A}_K$ of the form (2.8). Moreover, since v_K is a projection onto a convex set, it can be characterized by

$$x_K - \nabla_x \Phi_K - v_K = -(\nabla_x \Phi_K + d_K) \in \mathcal{N}_\Omega(v_K).$$

By the expression (3.25), $\mathcal{N}_\Omega(v_K)$ is trivially a subset of the row space of $\tilde{A}_K$ and the latter is also the orthogonal complement of the null space of $\tilde{A}_K$, i.e., $T(v_K)^\perp$. Because $T(v_K)^\perp$ is a vector space, we also have

$$\nabla_x \Phi_K + d_K \in T(v_K)^\perp,$$

and we obtain, by linearity of $P_{T(v_K)} [\cdot]$,

$$P_{T(v_K)} [\nabla_x \Phi_K + d_K] = 0 \iff P_{T(v_K)} [\nabla_x \Phi_K] = -P_{T(v_K)} [d_K]. \quad (3.28)$$

Combining (3.27) and (3.28) gives

$$\|P_{T(x_*)} [\nabla_x \Phi_K]\| \leq \|P_{T(v_K)} [\nabla_x \Phi_K]\| = \|P_{T(v_K)} [d_K]\| \leq \|d_K\| \leq \omega_K,$$

whenever $K \in \mathcal{K}$ is above the threshold K_0 , which is the desired inequality. $\square$

The next lemma is a standard result about the behavior of the Lagrange multipliers in AL methods.

Lemma 8 ([22], Lemma 14.4.1). *Assume that $\lim_{K \rightarrow \infty} \mu_K = \infty$ when Algorithm 1 is executed. Then*

$$\lim_{K \rightarrow \infty} \frac{\|\lambda_K\|}{\mu_K} = 0.$$

We now state the main convergence results. Their statement parallels [21, Lemma 4.3]. Since the framework presented there differs from ours in the criticality measure used, the arguments of the proof must be adapted.

Lemma 9. *Suppose that Assumptions 1 and 3 hold. Let $(x_K)_{K \in \mathcal{K}}$ be a sequence of iterates in Ω satisfying Assumption 8 and converging to x_* for which Assumption 9 holds. Let $\lambda_* = \lambda(x_*)$, $(\lambda_K)_{K \in \mathcal{K}}$ be any sequence of vectors and $(\mu_K)_{K \in \mathcal{K}}$ be a non-decreasing sequence of positive scalars. Assume that (2.6) holds with $\lim_{K \in \mathcal{K}} \omega_K = 0$.*

Then, there are positive constants κ_2, κ_3 such that

$$\|\bar{\lambda}_K - \lambda_*\| \leq \kappa_2 \omega_K + \kappa_3 \|x_K - x_*\|, \quad (3.29)$$

$$\|\lambda(x_K) - \lambda_*\| \leq \kappa_3 \|x_K - x_*\|, \quad (3.30)$$

and

$$\|c(x_K)\| \leq \kappa_2 \frac{\omega_K}{\mu_K} + \kappa_3 \frac{\|x_K - x_*\|}{\mu_K} + \frac{\|\lambda_K - \lambda_*\|}{\mu_K}. \quad (3.31)$$

If, in addition, $c(x_) = 0$, then x_* is a first-order critical point for problem (1.1) with corresponding Lagrange multipliers λ_* and with $\lim_{K \in \mathcal{K}} \bar{\lambda}_K = \lim_{K \in \mathcal{K}} \lambda(x_K) = \lambda_*$.*

Proof. By Assumptions 1 and 9, for K sufficiently large, the matrix $(C_K N_*)^\dagger$ is well defined, bounded and converges to $(C_* N_*)^\dagger$. Therefore, there is a constant κ_2 such that

$$\|((C_K N_*)^\dagger)^\top\| \leq \kappa_2.$$

Furthermore, by Lemma 7, we have

$$\|N_*^\top \nabla_x \Phi_K\| = \|P_{T(x_*)} [\nabla_x \Phi_K]\| \leq \omega_K,$$

for $K \in \mathcal{K}$ large enough. Thus we may deduce

$$\begin{aligned} \|\bar{\lambda}_K - \lambda(x_K)\| &= \|((C_K N_*)^\dagger)^\top N_*^\top \nabla f_K + \bar{\lambda}_K\| \\ &= \|((C_K N_*)^\dagger)^\top (N_*^\top \nabla f_K + (C_K N_*)^\top \bar{\lambda}_K)\| \\ &= \|((C_K N_*)^\dagger)^\top (N_*^\top \nabla_x \Phi_K)\| \\ &\leq \|((C_K N_*)^\dagger)^\top\| \|N_*^\top \nabla_x \Phi_K\| \\ &\leq \kappa_2 \omega_K. \end{aligned} \tag{3.32}$$

Furthermore, by the mean value theorem for vector-valued functions, we have

$$\lambda(x_K) - \lambda_* = \int_0^1 \nabla \lambda(x(t))(x_K - x_*) dt,$$

where $x(t) = x_K + t(x_* - x_K)$. By our discussion of the least-squares multipliers (3.26), one can show that the above integrand is bounded for x_K sufficiently close to x_* , hence

$$\|\lambda(x_K) - \lambda_*\| \leq \kappa_3 \|x_K - x_*\|,$$

for $K \in \mathcal{K}$ sufficiently large and for some positive constant κ_3 , giving (3.30). Then, combining the latter inequality and (3.32), we may deduce that

$$\begin{aligned} \|\bar{\lambda}_K - \lambda_*\| &\leq \|\bar{\lambda}_K - \lambda(x_K)\| + \|\lambda(x_K) - \lambda_*\| \\ &\leq \kappa_2 \omega_K + \kappa_3 \|x_K - x_*\|, \end{aligned}$$

which is inequality (3.29). Next, by expression (2.2), we have $c(x_K) = (\bar{\lambda}_K - \lambda_K)/\mu_K$, which leads to

$$\begin{aligned} \|c(x_K)\| &\leq \frac{\|\bar{\lambda}_K - \lambda_*\|}{\mu_K} + \frac{\|\lambda_K - \lambda_*\|}{\mu_K} \\ &\leq \kappa_2 \frac{\omega_K}{\mu_K} + \kappa_3 \frac{\|x_K - x_*\|}{\mu_K} + \frac{\|\lambda_K - \lambda_*\|}{\mu_K}, \end{aligned}$$

where we have used (3.29). This concludes the first part of the lemma.

Since $(\omega_K)_K$ tends to 0 and $(x_K)_{K \in \mathcal{K}}$ converges to x_* by assumption, inequalities (3.29) and (3.30) imply that both sequences $(\bar{\lambda}_K)_K$ and $(\lambda(x_K))_K$ converge to λ_* for $K \in \mathcal{K}$. Therefore, from identity (2.3), we obtain that the sequence of gradients $\nabla_x \Phi_K$ converges to $\nabla_x \mathcal{L}(x_*, \lambda_*)$.

Finally, assume that $c(x_*) = 0$. Because (2.6) holds, we have on the one hand that

$$\lim_{K \in \mathcal{K}} \|P_\Omega [x_K - \nabla_x \Phi_K] - x_K\| = 0. \tag{3.33}$$

On the other hand, the set Ω being closed and convex, the projection mapping $P_\Omega[\cdot]$ is continuous, so $P_\Omega[x_K - \nabla_x \Phi_K] \rightarrow P_\Omega[x_* - \nabla_x \mathcal{L}(x_*, \lambda_*)]$ as $K \in \mathcal{K}$ increases. With (3.33), we obtain

$$P_\Omega[x_* - \nabla_x \mathcal{L}(x_*, \lambda_*)] = x_*.$$

Therefore, we have established that (x_*, λ_*) is a KKT point and the lemma is proved. $\square$

We can now establish the global convergence of Algorithm 1.

Theorem 10. *Suppose that Assumptions 1 and 3 hold. Let $(x_K)_K$ be a sequence of iterates generated by Algorithm 1 satisfying Assumption 8. Further assume that there is a subsequence $(x_K)_{K \in \mathcal{K}}$ converging to x_* , for which Assumption 9 holds and let $\lambda_* = \lambda(x_*)$. Then, the conclusions of Lemma 9 hold.*

Proof. As in the proof of Lemma 9, the inequalities (3.29), (3.30), (3.31) can be derived from the assumptions. All that remains is to show that $c(x_*) = 0$. There are two cases to distinguish.

First, if the sequence of penalty parameters is bounded above, then there is K_0 sufficiently large such that $\|c(x_K)\| \leq \eta_K$ for all $K \geq K_0$. Since $\eta_K \rightarrow 0$, we get that $c(x_*) = 0$.

Next, if the sequence of penalty parameters is unbounded, then by the update rule, we have that $(\mu_K)_K$ grows to ∞ . Hence, by Lemma 8, the ratio λ_K/μ_K converges to zero. Therefore, by inequality (3.31), we also obtain that $c(x_*) = 0$. In both cases, x_* is feasible with respect to the nonlinear constraints and the remaining conclusions of Lemma 9 hold. $\square$

Our convergence analysis is based on the criticality measure (2.6) but, as we already mentioned in Section 2, the latter is not available in closed form for a general polyhedron such as (1.2). This is what justified our choice to monitor criticality in our algorithm through the reduced gradient norm $\|P_{T(x_K)}[\nabla_x \Phi_K]\|$, which only involves the projection onto the tangent space of the currently active bounds. The two measures are linked through the tangent cone $\mathcal{T}_\Omega(x_K)$, which is the dual cone of $\mathcal{N}_\Omega(x_K)$. Indeed, one always has

$$\|P_\Omega[x_K - \nabla_x \Phi_K] - x_K\| \leq \|P_{\mathcal{T}_\Omega(x_K)}[-\nabla_x \Phi_K]\|$$

and whenever the multipliers associated with the bounds active at x_K are of the correct sign, the projection onto the cone reduces to the projection onto $T(x_K)$, so that [21, Section 7.2]

$$\|P_{\mathcal{T}_\Omega(x_K)}[-\nabla_x \Phi_K]\| = \|P_{T(x_K)}[\nabla_x \Phi_K]\|.$$

Consequently, under the assumption that these sign conditions hold at every outer iterate, the reduced-gradient rule (2.10) certifies the criticality test (2.6), and Theorem 10 applies to the iterates the implementation produces. Without this assumption the reduced gradient controls only the tangential component of $\nabla_x \Phi_*$, and the limit point may not be critical.

4 Numerical experiments

In this section, we report numerical experiments conducted with the Julia implementation of our algorithm, named TRAULLS¹. All runs were performed with Julia 1.12.1 on a machine equipped with an Intel[®] Core[™] Ultra 9 processor 285 and 64 GiB of RAM. All the benchmarks we present here are also accessible in the `benchmark` folder of the GitHub repository. The tests

¹Trust Region AUgmented nonLinear Least-squares Solver

were carried out on a set of 52 constrained NLS problems from the collections [35, 41, 55], and problems 2 and 3 referenced in [12]. For problems with variable dimensions, we set the number of variables to $n \in \{100, 500, 1000\}$. Thus, we have a total of 82 problem instances ranging from 2 to 1001 variables, with up to 2994 residuals and 999 constraints. Problems with inequality constraints were converted to the form (1.1) by adding slack variables. For our tests, we set the criticality tolerance to $\omega_* = 10^{-5}$, the feasibility tolerance to $\eta_* = 10^{-6}$, the maximum number of outer iterations to 500 and the maximum number of inner iterations to 1000. The remaining parameters are set to $\mu_0 = 10$, $\tau = 100$, $\kappa_\omega = \beta_\omega = \eta = \omega = 1$, $\kappa_\eta = 0.1$, $\beta_\eta = 0.9$, $\kappa_{\text{hyb}} = 0.1$, $\delta_0 = 0.1$, and the trust-region constants $\alpha_1 = 0.25$, $\alpha_2 = 2.5$, $\eta_1 = 0.25$, $\eta_2 = 0.75$, $\gamma_1 = 0.0625$, in accordance with the conditions of Section 2.5.

To avoid any bias in computation time caused by Julia’s just-in-time compilation, and for all solvers considered in this section, each instance was solved once to warm up, the result of that first solve being discarded, then solved three times with the minimum of the three timings reported. We present our results with Dolan–Moré performance profiles [27] produced with the `BenchmarkProfiles.jl` package [49]. We exclude from the profiles the problems on which the solvers considered for a given comparison do not all converge to the same solution, in the sense that

$$\frac{f_{\max} - f_{\min}}{1 + \max(|f_{\max}|, |f_{\min}|)} > 10^{-2}, \quad (4.1)$$

where $f_{\max}$ and $f_{\min}$ are respectively the maximum and minimum values of the objective obtained on a given instance. A similar discarding heuristic is also used in [50, 62].

4.1 Comparison of Hessian approximations

Our first set of experiments compares the behavior and performance of our algorithm depending on the Hessian approximation used during the inner minimization. We compare the GN approximation (2.22) against structured approximations of the form (2.23). Based on the structured secant equation (2.26), we implemented the SR1 update we discussed and the BFGS update that one can derive similarly. Both structured approximations were used in their plain and hybrid variants with the switching rule (2.29).

4.1.1 Profiles on the whole test set

We first compare these five strategies with respect to three metrics: computation time, number of outer iterations and number of inner iterations. The last metric coincides with the number of residual evaluations, since the trust-region mechanism evaluates the residuals exactly once per inner iteration. Results are reported in Figure 2.

Two variants fail to reach a first-order critical point. The plain SR1 variant reaches the maximum number of outer iterations on problem HS373 and returns a point that is only feasible. The plain BFGS variant does the same on the instance of problem 5.1 from [41] with $n = 500$, although it attains there the same value of the objective as the other variants. Problem HS23 and the instances of problems 5.13 and 5.14 from [41] with $n = 100$ were removed after evaluating the relative-gap criterion (4.1). The profiles are therefore built on the remaining 79 instances, for which all five variants agree on the solution.

In terms of computation time, the GN approximation is the fastest and is also the cheapest per iteration because it requires no additional storage or update computation, and because our method mainly involves matrix-vector products. The GN approximation especially dominates

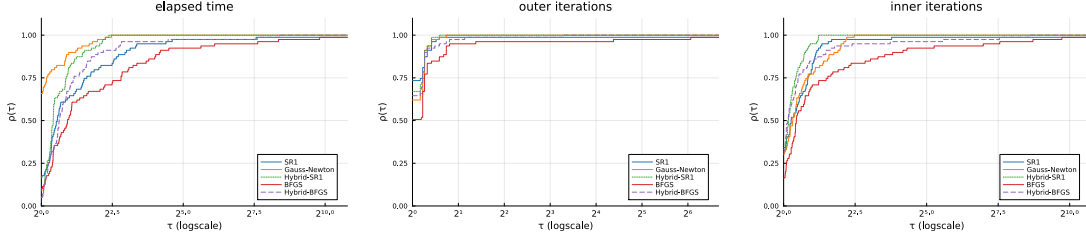


Figure 2: Performance profiles comparing five different Hessian approximation strategies with respect to computation time (left), number of outer iterations (center), and number of inner iterations (right).

on problems having sparse Jacobians and small residuals at the solution, since the second-order terms are still stored as a dense matrix. The Hybrid-SR1 variant is a close second.

Measured by outer iterations, the five strategies are almost indistinguishable, which is expected since the outer loop is governed by the augmented Lagrangian updates rather than by the Hessian model. The number of inner iterations, on the other hand, separates them clearly. There, the hybrid variants outperform their plain counterparts and the GN approximation alike. This is consistent with the discussion of the hybrid switching rule (2.29), which – as in the unconstrained case – correctly identifies situations where the Gauss–Newton approximation must be corrected and activates the structured quasi-Newton correction accordingly. Overall, Hybrid-SR1 appears to be the most robust variant, which also indicates that the SR1 approximation is well suited to handling negative curvature in the trust-region setting.

4.1.2 Profiles stratified by residual magnitude at the solution

To refine these observations and analyze separately the effectiveness of the approximate second-order correction and of the hybrid switching scheme, we stratify the previous profiles by the squared norm of the residuals at the solution, denoted $\|r_{\text{opt}}\|^2$. We distinguish four regimes: zero residuals ($\|r_{\text{opt}}\|^2 < 10^{-8}$, 18 instances), small residuals ($10^{-8} \leq \|r_{\text{opt}}\|^2 < 1$, 17 instances), medium residuals ($1 \leq \|r_{\text{opt}}\|^2 < 100$, 22 instances) and large residuals ($\|r_{\text{opt}}\|^2 \geq 100$, 22 instances). For this comparison, we retain the GN approximation and the SR1 update in its plain and hybrid variants. Figure 3 reports the profiles on the number of residual evaluations – equivalently, as discussed above, on the number of inner iterations – and Figure 4 those on computation time.

The profiles on residual evaluations display a clear trend as the residual magnitude grows. The GN approximation performs better on the zero-residual instances but worse on the nonzero-residual ones, whereas Hybrid-SR1 is consistent across all residual magnitudes. The gap between the two even widens as the residuals get larger. The hybrid scheme therefore matches GN when the residuals vanish at the solution and takes a decisive lead as soon as the residuals are significant. This is precisely the behavior the switching rule is designed to produce.

The plain SR1 update stays close to its hybrid counterpart on the small- and medium-residual instances, and is even ahead on the former. The second-order correction is thus beneficial in itself, and the hybrid rule does not always exploit it fully in this intermediate regime. A plausible explanation is that the switching test (2.29) monitors the relative decrease of the augmented Lagrangian, a quantity influenced by both the residual size and the constraint

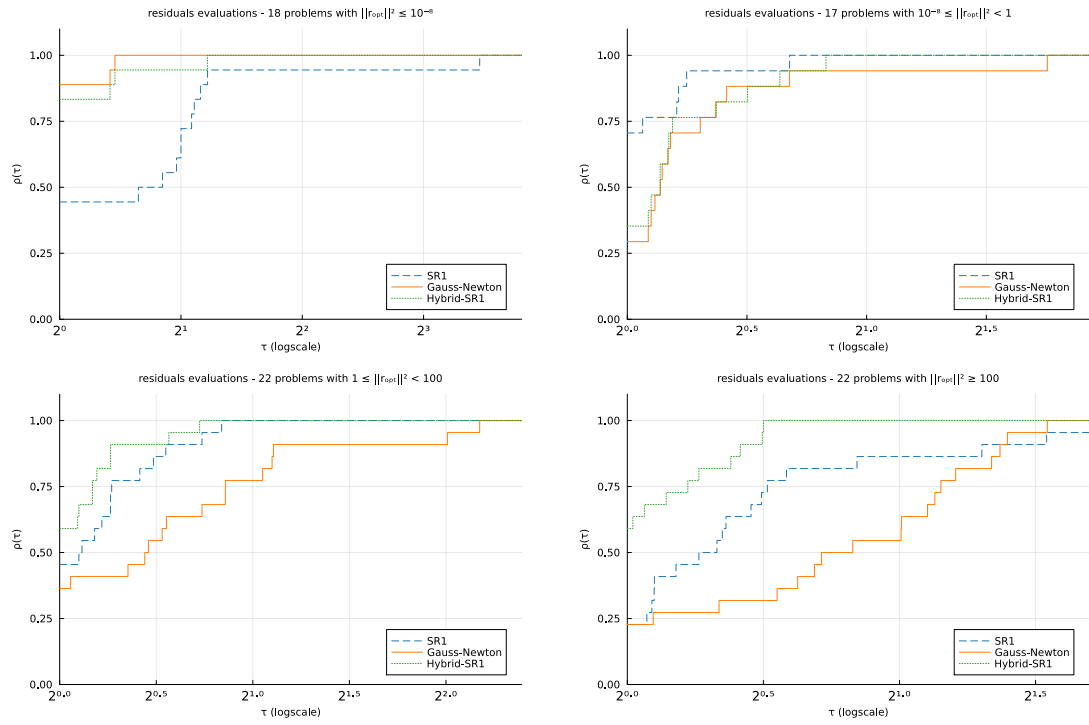


Figure 3: Performance profiles on the number of residual evaluations, stratified by the squared norm of the residuals at the solution: zero residuals (top left), small residuals (top right), medium residuals (bottom left) and large residuals (bottom right).

violation, so that nonzero but still small residuals are not always identified as such. In the large-residual regime, by contrast, the hybrid rule is unambiguously the better strategy.

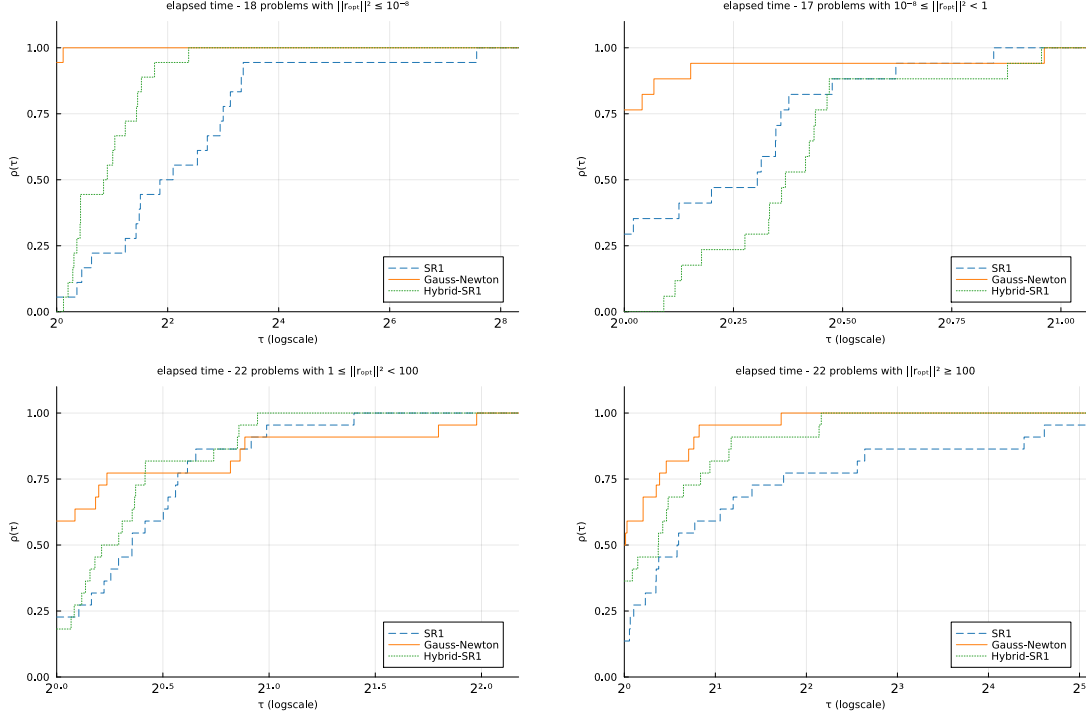


Figure 4: Performance profiles on computation time, stratified by the squared norm of the residuals at the solution: zero residuals (top left), small residuals (top right), medium residuals (bottom left) and large residuals (bottom right).

Computation time, however, favors the GN approximation in every stratum, even though the gap narrows as the residuals grow. The savings in residual evaluations achieved by the hybrid scheme therefore do not compensate for the additional cost of forming and using a dense second-order correction in the subsequent linear-algebra computations. This is largely due to how the test set is built. For all the problems considered, residuals are available by direct computation and their evaluation is consequently very cheap. In many applications, and in particular when the residuals come from a simulation code or from the solution of a system of ordinary differential equations, their evaluation is the dominant computational cost. In such settings, the profiles of Figure 3 are the relevant ones, and they show that the structured correction with hybrid switching does reduce the amount of work that actually matters.

To argue the latter observation, we ask at which evaluation cost the structured correction starts to pay off. Assume that one residual evaluation costs c more than it does here, every other cost being unchanged. The total time of a solver becomes

$$T(c) = T + cn_r^{\text{eval}}, \quad (4.2)$$

where T is the time we measured and n_r^{eval} the number of residual evaluations it performed. Comparing the GN approximation with Hybrid-SR1 on a given instance, the two are equally

fast at the break-even cost

$$c^* = \frac{T_{\text{hyb}} - T_{\text{gn}}}{n_{\text{gn}}^{\text{eval}} - n_{\text{hyb}}^{\text{eval}}}, \quad (4.3)$$

which is meaningful when the hybrid variant performs fewer residual evaluations but takes more time, the only regime in which a crossover exists. Beyond c^* , the hybrid variant is the faster of the two. This model keeps the iteration counts fixed and charges only the evaluations, so it describes a problem whose residuals are more expensive but whose solution path is unchanged.

Before applying it, we record in Table 1 how the evaluation counts of the two variants compare, stratum by stratum. The relative-gap criterion (4.1) is applied to the objective values reached by the GN approximation and Hybrid-SR1 alone, and it discards no instance. The pairwise counts are as informative as the geometric means. On the zero-residual instances the two variants perform exactly the same number of evaluations on 17 of the 18 instances, which confirms that the switching rule is inert when the correction is not needed and that it costs nothing there. The saving then grows with the residual magnitude. The Jacobian and gradient counts follow the same trend, which matters here, since an application whose residuals are expensive to evaluate usually has expensive derivatives as well.

stratum	instances	residual evaluations			geometric mean	
		fewer	equal	more	Jacobian	gradient
zero	18	0	17	1	1.04	1.04
small	17	4	7	6	0.95	0.96
medium	23	14	4	5	0.76	0.79
large	24	18	0	6	0.67	0.70

Table 1: Evaluation counts of Hybrid-SR1 relative to the GN approximation, by residual magnitude at the solution. The three middle columns count the instances on which Hybrid-SR1 performs fewer, exactly as many, or more residual evaluations than GN. The last two columns report the geometric mean of the ratio of the Jacobian and gradient evaluation counts, a value below one favoring Hybrid-SR1.

We now restrict the analysis to the 47 medium- and large-residuals since these are the ones on which the correction is meant to act. The median break-even cost (4.3) is 0.286 milliseconds, with a first quartile of 0.003 and a third quartile of 1.359. In other words, it is enough that a residual evaluation costs a few tenths of a millisecond in total for Hybrid-SR1 to become the faster variant on the median instance. An evaluation that requires a simulation run or the integration of a system of ordinary differential equations is two to three orders of magnitude above that threshold.

Figure 5 shows the proportion of these instances on which Hybrid-SR1 is faster, as a function of the extra cost c . It starts with the times we measured and then saturates slightly below 70% as c increases. The remaining instances are those on which the hybrid variant does not perform fewer residual evaluations, and no evaluation cost can make it faster on them. The conclusion we draw is therefore a conditional one, namely that the structured correction is the better choice on the majority of the problems with nonzero residuals as soon as the residuals are genuinely expensive, and that the threshold at which this happens is low enough to be reached by most applications in which the residuals come from a simulation.

Based on these results, we retain the Hybrid-SR1 variant as the best overall strategy for TRAULS, and use it in the comparisons that follow.

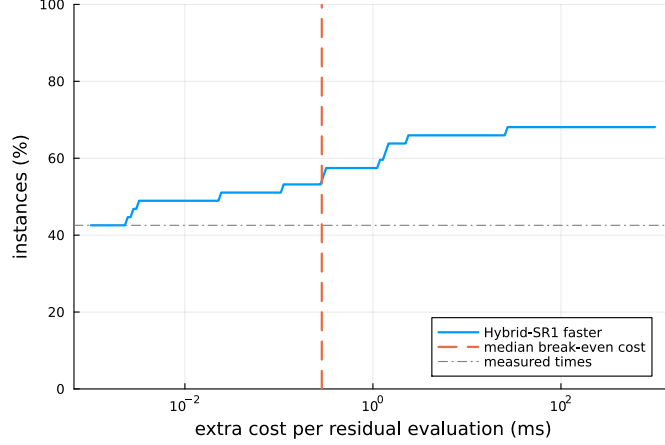


Figure 5: Proportion of the 47 medium- and large-residual instances on which Hybrid-SR1 is faster than the GN approximation, as a function of the extra cost charged to every residual evaluation in model (4.2). The dotted line marks the median cost of a residual evaluation in the test set and the dashed line the median break-even cost (4.3).

4.2 Comparison with IPOPT and Percival

We now compare TRAULLS (with the Hybrid-SR1 approximation) against the solvers IPOPT [62] and Percival [3]. The latter is also an augmented Lagrangian algorithm and has a native Julia implementation. Its inner minimization uses a Julia version of the TRON [40] algorithm – part of the `JSOSolvers.jl` package [45]. To benefit from the benchmarking facilities provided by the JuliaSmoothOptimizers (JSO)² ecosystem, we used the wrapper `NLPModelsIpopt.jl` [58]. This allows us to run both reference solvers on the same test set as TRAULLS – all models being encoded in `NLSProblems.jl` [56] – and to export the benchmark metrics using `SolverBenchmark.jl` [57]. All solvers shared the experimental setup and the tolerances described at the beginning of this section, and were additionally given a maximum CPU time of 10 minutes per instance, a limit consistent with the dimensions of the problems considered.

To make the comparison with IPOPT relevant, we set the parameters `nlp_scaling_method = none`, `dual_inf_tol = Inf`, `constr_viol_tol = Inf`, `compl_inf_tol = Inf`, `acceptable_iter = 0` and the maximum number of iterations to 1000. We also set `hessian_approximation = ‘‘limited-memory’’`, so that IPOPT builds its Hessian model with an L-BFGS update of memory 6 instead of exact second derivatives. This choice makes the comparison meaningful with respect to the derivative information used. For Percival, since its outer iteration is the same as that of Algorithm 1, we set its tolerances and parameters to the same values as ours. We note that Percival, through its TRON inner solver, uses exact Hessian-vector products, and is therefore the only solver of the three that accesses second derivatives.

For these tests, we compare the three solvers with respect to the computation time, the number of residual evaluations and the number of gradient evaluations. After evaluating criterion (4.1), we removed from the profiles the following nine instances: HS61, HS77, HS264, the instances with $n = 100$ of problems 5.1, 5.4, 5.13 and 5.14 from [41], and the instances with

²<https://jso.dev>

$n = 500$ and $n = 1000$ of problem 5.16 from the same collection. The profiles of Figure 6 are thus built on the remaining 73 instances. On these, TRAULLS and IPOPT converge on every instance, while Percival reaches the maximum CPU time on the two instances of problem 5.11 from [41] with $n = 500$ and $n = 1000$. As a side note, we also ran IPOPT with exact second derivatives, and this configuration failed on two instances, reaching the maximum number of iterations on HS27 and terminating with a restoration-phase failure on HS70.

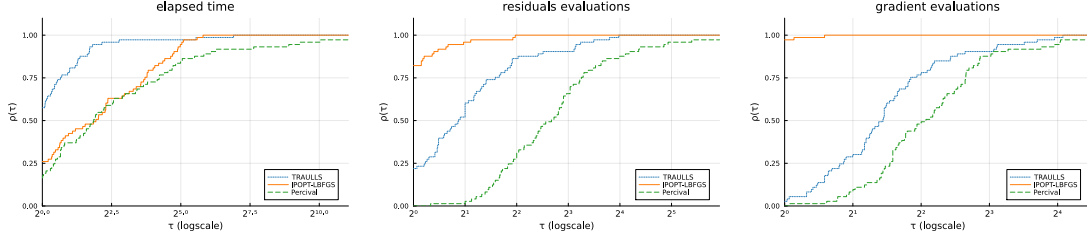


Figure 6: Performance profiles comparing TRAULLS (Hybrid-SR1), IPOPT, and Percival on the test set, with respect to computation time (left), residual evaluations (center), and gradient evaluations (right).

In terms of computation time, TRAULLS is the fastest solver on more than 50% of the instances. This advantage is, however, strongly dependent on the problem size. For instance, restricting the comparison to IPOPT, TRAULLS is faster on 40 of the 41 instances with $n \leq 100$, but on only 8 of the 26 instances with $n \geq 500$, where our computation time is on average – in the geometric mean sense – 1.79 times that of IPOPT. The margin observed on the smallest instances should be read with care, as both codes solve them in a few tens of milliseconds at most, with median times of 0.15 and 1.5 milliseconds respectively, so that the ratio there mostly reflects a lower fixed cost per solve rather than a difference in the optimization work performed. The large instances are where the gap is algorithmically meaningful, and there the cause is the dense storage and manipulation of the structured second-order correction, whose cost grows quadratically with the number of variables while the rest of the method only involves matrix-vector products. The example of problem 5.12 from [41] with $n = 1000$ illustrates that point: Hybrid-SR1 requires 142 inner iterations against 354 for the GN approximation, yet takes 2.21 seconds against 1.71. This is the limitation we discuss in the conclusion, and it is the reason a limited-memory form of the structured update is the natural next step for large-scale problems.

Regarding the evaluation metrics, IPOPT dominates both profiles – with a larger gap on gradients than on residuals – and this dominance decomposes into two distinct effects. TRAULLS performs on average 1.91 times as many residual evaluations as IPOPT, which reflects a genuine difference in the number of steps needed. On top of that, the two methods do not evaluate gradients at the same rate, since our trust-region scheme performs one gradient evaluation per inner iteration, whereas the line search in IPOPT reuses a single gradient across the trial points of a backtracking sequence. Part of the gap is thus structural, inherent to the globalization strategy rather than to the quality of our Hessian model.

These results should be read with the role of each solver in mind. IPOPT is a mature interior-point implementation, and it is included here as an external reference point rather than as the target of our contribution. The comparison that isolates what this work proposes is the one with Percival, which follows the same augmented Lagrangian framework, is run with

the same tolerances and parameters, and differs from TRAULLS essentially in that it treats the problem as a generic nonlinear program. There, the advantage of TRAULLS is unambiguous and holds on all three metrics. In geometric mean over the 73 retained instances, Percival takes 3.67 times as long as TRAULLS, performs 2.94 times as many residual evaluations and 1.47 times as many gradient evaluations. Also, unlike the comparison with IPOPT, this advantage does not rest on the smallest instances; it is moreover obtained even though TRAULLS only approximates second-order information, whereas Percival evaluates exact Hessian-vector products. Within a common algorithmic framework, exploiting the least-squares structure through the Gauss–Newton term and its structured correction therefore leads to a clear improvement, which is the claim this paper makes. That TRAULLS is at the same time faster than IPOPT on most of the test set, while requiring more function evaluations, situates our implementation as a competitive solver whose remaining bottleneck – the dense second-order correction on large instances – is identified and addressable.

4.3 Reproducibility

Every number reported in this section can be regenerated from the repository. The `benchmark` folder contains a Julia environment which pins every package involved to the version used for the numerical experiments. The script `run_experiments.jl` runs the solvers under the protocol described at the beginning of this section and writes one result file per solver, recording for each instance the objective value reached, the elapsed time, the evaluation counters, the iteration counts and the termination status. Those files are the ones shipped in `results`, so that the profiles and the tables can be rebuilt without running the solvers again. The notebook `profiles.ipynb` produces the performance profiles and `residual_cost_analysis.jl` the break-even analysis of Section 4.1. The input models for TRAULLS are given in the `traulls_models` folder.

Conclusion

In this paper, we have presented an algorithm able to solve a class of problems, namely NLS objectives under nonlinear equalities combined with bounds and linear equalities kept explicit, for which no dedicated method was available. Our method fully exploits the structure of the problem through the AL reformulation and a structured Hessian approximation. The resulting implementation is faster and more robust than a traditional AL solver, and remains competitive in computation time with a state-of-the-art solver for nonlinear programming. Despite this promising performance, the main limitation for large-scale applications lies in the dense storage of the structured second-order correction of the Hessian, in addition to the subsequent computations that become prohibitive as the number of variables grows. A limited-memory variant of the structured update, following [14], would be a natural feature to implement. To tackle problems with a large number of residuals, another avenue of research could be to combine the AL framework with an adaptive sampling strategy applied to the residuals, lowering the dimensions of the inner minimization subproblems. This would extend to the constrained case techniques currently investigated for unconstrained problems [8].

Acknowledgments: The first author is supported by Mitacs and Hydro-Québec through grants IT40867 and IT38322. The authors would also like to thank the Natural Sciences and

Engineering Research Council of Canada (NSERC), which supports the second author through a Discovery Grant, and the Institute for Data Valorization (IVADO).

References

- [1] M. Al-Baali and R. Fletcher. Variational methods for non-linear least-squares. *The Journal of the Operational Research Society*, 36(5):405–421, 1985. doi: 10.1057/jors.1985.68.
- [2] R. Andreani, E.G. Birgin, J.M. Martinez, and M.L. Schuverdt. On augmented Lagrangian methods with general lower-level constraints. *SIAM Journal on Optimization*, 18(4):1286–1309, 2008. doi: 10.1137/060654797.
- [3] S. Arreckx, A. Lambe, J.R.R.A. Martins, and D. Orban. A matrix-free augmented Lagrangian algorithm with application to large-scale structural design optimization. *Optimization and Engineering*, 17:359–384, 2016. doi: 10.1007/s11081-015-9287-9.
- [4] M.C. Bartholomew-Biggs. The estimation of the hessian matrix in nonlinear least-squares problems with non-zero residuals. *Mathematical Programming*, 12:67–80, 1977. doi: 10.1007/BF01593770.
- [5] S. Bellavia, M. Macconi, and B. Morini. STRSCNE: A scaled trust-region solver for constrained nonlinear equations. *Computational Optimization and Applications*, 28:31–50, 2004.
- [6] S. Bellavia, J. Gondzio, and B. Morini. Computational experience with numerical methods for nonnegative least-squares problems. *Numerical Linear Algebra with Applications*, 18(3):363–385, 2011.
- [7] S. Bellavia, S. Gratton, and E. Riccietti. A Levenberg-Marquardt method for large nonlinear least-squares problems with dynamic accuracy in functions and gradients. *Numerische Mathematik*, 140:791–825, 2018. doi: 10.1007/s00211-018-0977-z.
- [8] Stefania Bellavia, Greta Malaspina, and Benedetta Morini. A variable dimension sketching strategy for nonlinear least-squares. *SIAM Journal on Scientific Computing*, 48(3):A1206–A1234, 2026. doi: 10.1137/25M175980X.
- [9] E.H. Bergou, Y. Diouane, V. Kungurtsev, and C.W. Royer. A nonmonotone matrix-free algorithm for nonlinear equality-constrained least-squares problems. *SIAM Journal on Scientific Computing*, 43(5):S743–S766, 2021. doi: 10.1137/20M1349138.
- [10] J.T. Betts. Solving the nonlinear least square problem: Application of a general method. *Journal of Optimization Theory and Applications*, 18:469–483, 1976. doi: 10.1007/BF00932656.
- [11] J. Bezanson, A. Edelman, S. Karpinski, and V.B. Shah. Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1):65–98, 2017. doi: 10.1137/141000671.
- [12] L.T. Biegler, J. Nocedal, C. Schmid, and D. Ternet. Numerical experience with a reduced hessian method for large scale constrained optimization. *Computational Optimization and Applications*, 15:45–67, 2000. doi: 10.1023/A:1008723031056.

- [13] Åke Björck. *Numerical Methods for Least Squares Problems*. SIAM, Philadelphia, PA, USA, 2nd edition, 2024. doi: 10.1137/1.9781611977950.
- [14] R.H. Byrd, J. Nocedal, and R.B. Schnabel. Representations of quasi-Newton matrices and their use in limited memory methods. *Mathematical Programming*, 63:129–156, 1994. doi: 10.1007/BF01582063.
- [15] A.R. Conn, N.I.M. Gould, and Ph.L. Toint. Global convergence of a class of trust region method algorithms for optimization with simple bounds. *SIAM Journal on Numerical Analysis*, 25(2):433–460, 1988. doi: 10.1137/0725029.
- [16] A.R. Conn, N.I.M. Gould, and Ph.L. Toint. Testing a class of methods for solving minimization problems with simple bounds on the variables. *Mathematics of Computation*, 50(182):399–430, 1988. doi: 10.1090/S0025-5718-1988-0929544-3.
- [17] A.R. Conn, N.I.M. Gould, and Ph.L. Toint. A globally convergent augmented Lagrangian algorithm for optimization with general constraints and simple bounds. *SIAM Journal on Numerical Analysis*, 28(2):545–572, 1991. doi: 10.1137/0728030.
- [18] A.R. Conn, N.I.M. Gould, and Ph.L. Toint. Convergence of quasi-Newton matrices generated by the symmetric rank one update. *Mathematical Programming*, 50:177–195, 1991. doi: 10.1007/BF01594934.
- [19] A.R. Conn, N.I.M. Gould, and Ph.L. Toint. *LANCELOT: A Fortran Package for Large-Scale Nonlinear Optimization (Release A)*. Springer Series in Computational Mathematics. Springer Berlin, Heidelberg, 1992. doi: 10.1007/978-3-662-12211-2.
- [20] A.R. Conn, N. Gould A. Sartenaer, and Ph.L. Toint. Global convergence of a class of trust region algorithms for optimization using inexact projections on convex constraints. *SIAM Journal on Optimization*, 3(1):164–221, 1993. doi: 10.1137/0803009.
- [21] A.R. Conn, N. Gould A. Sartenaer, and Ph.L. Toint. Convergence properties of an augmented Lagrangian algorithm for optimization with a combination of general equality and linear constraints. *SIAM Journal on Optimization*, 6(3):674–703, 1996. doi: 10.1137/S1052623493251463.
- [22] A.R. Conn, N.I.M. Gould, and Ph.L. Toint. *Trust Region Methods*. SIAM, Philadelphia, PA, USA, 2000. doi: 10.1137/1.9780898719857.
- [23] F.E. Curtis, H. Jiang, and D.P. Robinson. An adaptive augmented Lagrangian method for large-scale constrained optimization. *Mathematical Programming, Series A*, 152:201–245, 2015. doi: 10.1007/s10107-014-0784-y.
- [24] F. Delbos, J.Ch. Gilbert, R. Glowinski, and D. Sinoquet. Constrained optimization in seismic reflection tomography: a Gauss-Newton augmented Lagrangian approach. *Geophysical Journal International*, 164:670–684, 2006. doi: 10.1111/j.1365-246X.2005.02729.x.
- [25] J.E. Dennis Jr and R.B. Schnabel. *Numerical Methods for Unconstrained optimization and Nonlinear Equations*. Classics in Applied Mathematics. SIAM, Philadelphia, PA, USA, 1996. doi: 10.1137/1.9781611971200.

- [26] J.E. Dennis Jr, D.M. Gay, and R.E. Walsh. An adaptive nonlinear least-squares algorithm. *ACM Transactions on Mathematical Software*, 7(3):348–368, 1981. doi: 10.1145/355958.355965.
- [27] E.D. Dolan and J.J. Moré. Benchmarking optimization software with performance profiles. *Mathematical Programming, Series A* 91(2):201–213, 2002. doi: 10.1007/s101070100263.
- [28] R. Fletcher and C. Xu. Hybrid methods for nonlinear least squares. *IMA Journal of Numerical Analysis*, 7:373–389, 1987. doi: 10.1093/imanum/7.3.371.
- [29] P.E. Gill and D.P. Robinson. A primal-dual augmented Lagrangian. *Computational Optimization and Applications*, 51:1–25, 2012. doi: 10.1007/s10589-010-9339-1.
- [30] G.H. Golub and C.F. Van Loan. *Matrix Computations*. JHU Press, Maryland, MD, USA, 4th edition, 2013. doi: 10.56021/9781421407944.
- [31] M.L.N. Gonçalves and T.C. Menezes. Gauss–Newton methods with approximate projections for solving constrained nonlinear least squares problems. *Journal of Complexity*, 58: 101459, 2020. doi: 10.1016/j.jco.2020.101459.
- [32] N.I.M. Gould, M.E. Hribar, and J. Nocedal. On the solution of equality constrained quadratic programming problems arising in optimization. *SIAM Journal on Scientific Computing*, 23(4):1376–1395, 2001. doi: 10.1137/S1064827598345667.
- [33] Michel Grenier, J. Gagnon, C. Mercier, and J. Richard. Short-term load forecasting at Hydro-Québec TransÉnergie. In *2006 IEEE Power Engineering Society General Meeting*, Montreal, QC, Canada, 2006. doi: 10.1109/PES.2006.1709029.
- [34] M.R. Hestenes. Multiplier and gradient methods. *Journal of Optimization Theory and Applications*, 4:303–320, 1969. doi: 10.1007/BF00927673.
- [35] W. Hock and K. Schittkowski. *Test Examples for Nonlinear Programming Codes*, volume 187 of *Lecture Notes in Economics and Mathematical Systems*. Springer Berlin, Heidelberg, 2nd edition, 1980. doi: 10.1007/978-3-642-48320-2.
- [36] J. Hushens. On the use of product structure in secant methods for nonlinear least squares problems. *SIAM Journal on Optimization*, 4(1):108–129, 1994. doi: 10.1137/0804005.
- [37] J.E. Dennis Jr, H.J. Martinez, and R.A. Tapia. Convergence theory for the structured BFGS method with an application to nonlinear least squares. *Journal of Optimization Theory and Applications*, 61(2):161–178, 1999. doi: 10.1007/BF00962795.
- [38] K. Levenberg. A method for the solution of certain non-linear problems in least squares. *Quarterly of Applied Mathematics*, 2:164–168, 1944. doi: 10.1090/qam/10666.
- [39] Z. Li, M. Osborne, and T. Prvan. Adaptive algorithm for constrained least-squares problems. *Journal of Optimization Theory and Applications*, 114:423–441, 2002. doi: 10.1023/A:1016043919978.
- [40] C.-J. Lin and J.J. Moré. Newton’s method for large bound-constrained optimization problems. *SIAM Journal on Optimization*, 9(4):1100–1127, 1999. doi: 10.1137/S1052623498345075.

- [41] L. Lukšan and J. Vlček. Sparse and partially separable test problems for unconstrained and equality constrained optimization. Technical Report 767, Institute of Computer Science, Academy of Sciences of the Czech Republic, 1999.
- [42] L. Lukšan, C. Matonoha, and J. Vlček. Hybrid methods for nonlinear least squares problems. Technical Report 1246, Institute of Computer Science, Academy of Sciences of the Czech Republic, 2019.
- [43] N. Mahdavi-Amiri and R.H. Bartels. Constrained nonlinear least squares: An exact penalty approach with projected structured quasi-Newton update. *ACM Transactions on Mathematical Software*, 15(3):220–242, 1989. doi: 10.1145/66888.66891.
- [44] D.W. Marquardt. An algorithm for least squares estimation of non-linear parameters. *SIAM Journal*, 11:431–441, 1963. doi: 10.1137/0111030.
- [45] T. Migot, D. Monnet, D. Orban, and S. Abel Soares. JSOSolvers.jl: Unconstrained and bound-constrained optimization solvers. *Journal of Open Source Software*, 11(117):9467, 2026. doi: 10.21105/joss.09467. URL <https://doi.org/10.21105/joss.09467>.
- [46] J.J. Moré and G. Toraldo. On the solution of large quadratic programming problems with bound constraints. *SIAM Journal on Optimization*, 1(1):93–113, 1991. doi: 10.1137/0801008.
- [47] B.A. Murtagh and M.A. Saunders. Large-scale linearly constrained optimization. *Mathematical Programming*, 14:41–72, 1978. doi: 10.1007/BF01588950.
- [48] J. Nocedal and S.J. Wright. *Numerical Optimization*. Springer series in Operation Research and Financial Engineering. Springer, New York, NY, USA, 2nd edition, 2006. doi: 10.1007/978-0-387-40065-5.
- [49] D. Orban. BenchmarkProfiles.jl, 2019. URL <https://doi.org/10.5281/zenodo.4630955>.
- [50] D. Orban and A.S. Siqueira. A regularization method for constrained nonlinear least squares. *Computational Optimization and Applications*, 76(3):961–989, 2020. doi: 10.1007/s10589-020-00201-2.
- [51] M. Porcelli. On the convergence of an inexact Gauss-Newton trust-region method for nonlinear least-squares problems with simple bounds. *Optimization Letters*, 7:447–465, 2013. doi: 10.1007/s11590-011-0430-z.
- [52] M.J.D. Powell. A method for nonlinear constraints in minimization problems. In R. Fletcher, editor, *Optimization*, pages 283–298. Academic Press, London, 1969.
- [53] H. Pu, Z. Chen, B. Wang, and W. Xia. Constrained least squares algorithms for nonlinear unmixing of hyperspectral imagery. *IEEE Transactions on Geoscience and Remote Sensing*, 53(3):1287–1303, 2015. doi: 10.1109/TGRS.2014.2336858.
- [54] R.T. Rockafellar. The multiplier method of Hestenes and Powell applied to convex programming. *Journal of Optimization Theory and Applications*, 12(6):555–562, 1973. doi: 10.1007/BF00934777.

- [55] K. Schittkowski. *More Test Examples for Nonlinear Programming Codes*. Lecture Notes in Economics and Mathematical Systems. Springer Berlin, Heidelberg, 1987. doi: 10.1007/978-3-642-61582-5.
- [56] A. Soares Siqueira and D. Orban. NLSProblems.jl, 2019. URL <https://doi.org/10.5281/zenodo.4605405>.
- [57] A. Soares Siqueira and D. Orban. SolverBenchmark.jl, 2020. URL <https://doi.org/10.5281/zenodo.3948381>.
- [58] A. Soares Siqueira and D. Orban. NLPModelsIpopt.jl, 2026. URL <https://doi.org/10.5281/zenodo.19420624>.
- [59] R. Tapia. On secant updates for use in general constrained optimization. *Mathematics of Computations*, 51(183):181–202, 1988. doi: 10.2307/2008585.
- [60] I.B. Tjoa and L.T. Biegler. Simultaneous solution and optimization strategies for parameter estimation of differential-algebraic equation systems. *Industrial & Engineering Chemistry Research*, 30(2):376–385, 1991. doi: 10.1021/ie00050a015.
- [61] Ph.L. Toint and D. Tuytens. LSNNO, a FORTRAN subroutine for solving large-scale nonlinear network optimization problems. *ACM Transactions on Mathematical Software*, 18(3):308–328, 1992. doi: 10.1145/131766.131771.
- [62] A. Wächter and L.T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming, Series A*, 106:25–57, 2006. doi: 10.1007/s10107-004-0559-y.
- [63] H. Yabe and T. Takahashi. Factorized quasi-Newton methods for nonlinear least squares problems. *Mathematical Programming*, 51:75–100, 1991. doi: 10.1007/BF01586927.
- [64] J.Z. Zhang, L.H. Chen, and N.Y. Deng. A family of scaled factorized Broyden-like methods for nonlinear least squares problems. *SIAM Journal on Optimization*, 10(4):1163–1179, 2000. doi: 10.1137/S105262349834530.
- [65] W. Zhou and X. Chen. Global convergence of a new hybrid Gauss-Newton structured BFGS method for nonlinear least squares problems. *SIAM Journal on Optimization*, 20(5):2422–2441, 2010. doi: 10.1137/090748470.

Appendix

A Cauchy point computation

For the sake of clarity, we omit the iteration index during the rest of this appendix. We describe the procedure used to compute the first local minimizer of the quadratic $q(s) = \frac{1}{2} \langle s, Hs \rangle + \langle g, s \rangle$ along the projected gradient path $s(t) = P_\Omega [x - tg] - x$ with $t \geq 0$. This is an adaptation of Algorithm 17.3.1 from [22] to the case where linear equalities are present. For each $t \geq 0$, the projected step $s(t)$ satisfies the constraints

$$As(t) = 0, \quad s^{(l)} \leq s(t) \leq s^{(u)},$$

where matrix A is $m \times n$ with $m < n$ and vectors $s^{(l)}, s^{(u)}$ are iteration-dependent bounds having finite components. The breakpoints

$$0 = t_0 < t_1 < \dots < t_p,$$

correspond to the successive scalars at which at least one component of $s(t)$ satisfies a bound with equality. Assuming A has full row rank, there are $n - m$ degrees of freedom remaining and thus at most $n - m$ breakpoints before the projected gradient path is constant. We recall the recursive expression

$$s(t) = -(t - t_i)z_i + s(t_i),$$

for $t \in [t_i, t_{i+1})$ and with the reduced gradient z_i given by (3.5).

To find the first local minimum of the scalar function $\psi(t) = q(s(t))$, we successively study each interval $[t_i, t_{i+1})$ to determine whether or not it contains a local minimizer. Assume we have not found a local minimizer on $[t_0, t_i)$ and thus look at the interval $[t_i, t_{i+1})$. The model along this arc can be written

$$\psi(t) = \frac{\psi''_i}{2}(\Delta t)^2 + \psi'_i \Delta t \tag{A.1}$$

with $\psi''_i = \langle z_i, H z_i \rangle$, $\psi'_i = -\langle s(t_i), H z_i \rangle - \langle g, z_i \rangle$ and $\Delta t = t - t_i$. Different cases can occur depending on the values of the slope ψ'_i and the curvature ψ''_i . First, if

$$\begin{aligned} &\psi'_i > 0 \text{ or} \\ &(\psi'_i = 0 \text{ and } \psi''_i > 0), \end{aligned}$$

then t_i is the required minimizer. Next, if

$$\psi'_i < 0 \text{ and } \psi''_i > 0,$$

the quadratic (A.1) has a strict minimizer at

$$t_i - \frac{\psi'_i}{\psi''_i}.$$

If the latter belongs to the interval of interest, i.e.,

$$t_i - \psi'_i / \psi''_i < t_{i+1},$$

then it is the required minimizer. In other cases, the minimizer is at or beyond t_{i+1} . To prepare for the study of the next interval, we first need to find the next breakpoint, given by

the smallest scalar $t_{i+1} > t_i$ such that, at $s(t_{i+1})$, one of the free components hits one of its bounds. By introducing

$$\delta_i^- = \min_{(z_i)_j > 0} \left\{ \frac{(s(t_i))_j - s_j^{(l)}}{(z_i)_j} \right\}, \quad \delta_i^+ = \min_{(z_i)_j < 0} \left\{ \frac{(s(t_i))_j - s_j^{(u)}}{(z_i)_j} \right\},$$

the next breakpoint is given by

$$t_{i+1} = t_i + \delta_i, \tag{A.2}$$

with $\delta_i = \min(\delta_i^-, \delta_i^+)$. We then add the corresponding variable index to the list of fixed components, compute the next reduced gradient z_{i+1} and finally update the slope and curvature of the model along the next interval. A last termination case can occur. Since A is of rank m , at most $n - m$ bounds can become active, so if we never find a local minimum, the procedure still ends whenever it reaches the last breakpoint t_p . Indeed, past this breakpoint, the model along the projected gradient path is constant so we can return the last accumulated step $s(t_p)$ as the Cauchy step. Note that, in this case, it is also the total step, because no more variables can be modified. The finiteness of the box $[s^{(l)}, s^{(u)}]$ guarantees that all components of the projected step $s(t)$ are fixed at one of their bounds whenever t passes the last breakpoint. The procedure for the Cauchy step computation is outlined in Algorithm 4.

Algorithm 4 Cauchy step computation

Require: Hessian H , gradient g , bounds on the direction $s^{(l)}, s^{(u)}$

- 1: Identify $\mathcal{A}(0)$ and compute the direction z_0
 - 2: Set **found** $\leftarrow$ **false**
 - 3: Set $t_0 \leftarrow 0$, $s^{(0)} \leftarrow 0$ and initialize counter $i \leftarrow 0$
 - 4: **repeat**
 - 5: $\psi'_i \leftarrow -\langle s^{(i)}, H z_i \rangle - \langle g, z_i \rangle$, $\psi''_i \leftarrow \langle z_i, H z_i \rangle$
 - 6: Find the next breakpoint t_{i+1} by (A.2)
 - 7: **if** $\psi'_i > 0$ or ($\psi'_i = 0$ and $\psi''_i > 0$) **then**
 - 8: Set $s^C \leftarrow s^{(i)}$
 - 9: **found** $\leftarrow$ **true**
 - 10: **else if** $\psi'_i < 0$ and $\psi''_i > 0$ and $t_i - \psi'_i/\psi''_i < t_{i+1}$ **then**
 - 11: Set $\Delta t \leftarrow -\psi'_i/\psi''_i$
 - 12: Set $s^C \leftarrow s^{(i)} - \Delta t z_i$, **found** $\leftarrow$ **true**
 - 13: **else**
 - 14: Set $\Delta t \leftarrow t_{i+1} - t_i$
 - 15: Compute the next direction z_{i+1}
 - 16: Set $s^{(i+1)} \leftarrow s^{(i)} - \Delta t z_i$
 - 17: **end if**
 - 18: Increment $i \leftarrow i + 1$
 - 19: **if** $|\mathcal{A}(t_i)| = n - m$ **then**
 - 20: Set $s^C \leftarrow s^{(i)}$, **found** $\leftarrow$ **true**
 - 21: **end if**
 - 22: **until found**
 - 23: **return** s^C
-